

Departamento de Eléctrica, Electrónica y Telecomunicaciones

Nombre:

Génesis Alarcón

Tema:

Ejercicios de programación y algoritmia

NRC:

36466

Carrera:

Telecomunicaciones

Docente:

Ing. Jenny Alexandra Ruiz Robalino

24 de octubre del 2026

1- Media de dos valores

Desarrolla un programa que ingrese dos números por teclado e imprima su media. Realiza y completa la tabla de objetos del problema 1.1:

Análisis del problema

- **Entrada:** Dos números ingresados por teclado.
- **Proceso:** Sumar los dos números y dividir el resultado para 2.
- **Salida:** Mostrar la media aritmética de los dos números.
- **Restricciones:** Los datos ingresados deben ser números reales.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	num1	Primer número asignado	Real
Dato de entrada	num2	Segundo número asignado	Real
Dato de salida	media	Media de dos valores	Real

Seudocódigo

```
Algoritmo MediaDeDosValores
        // Definición de variables
        Definir num1, num2, media Como Real

        // Entrada de datos
        Escribir "Ingrese el primer numero:"
        Leer num1

        Escribir "Ingrese el segundo numero:"
        Leer num2

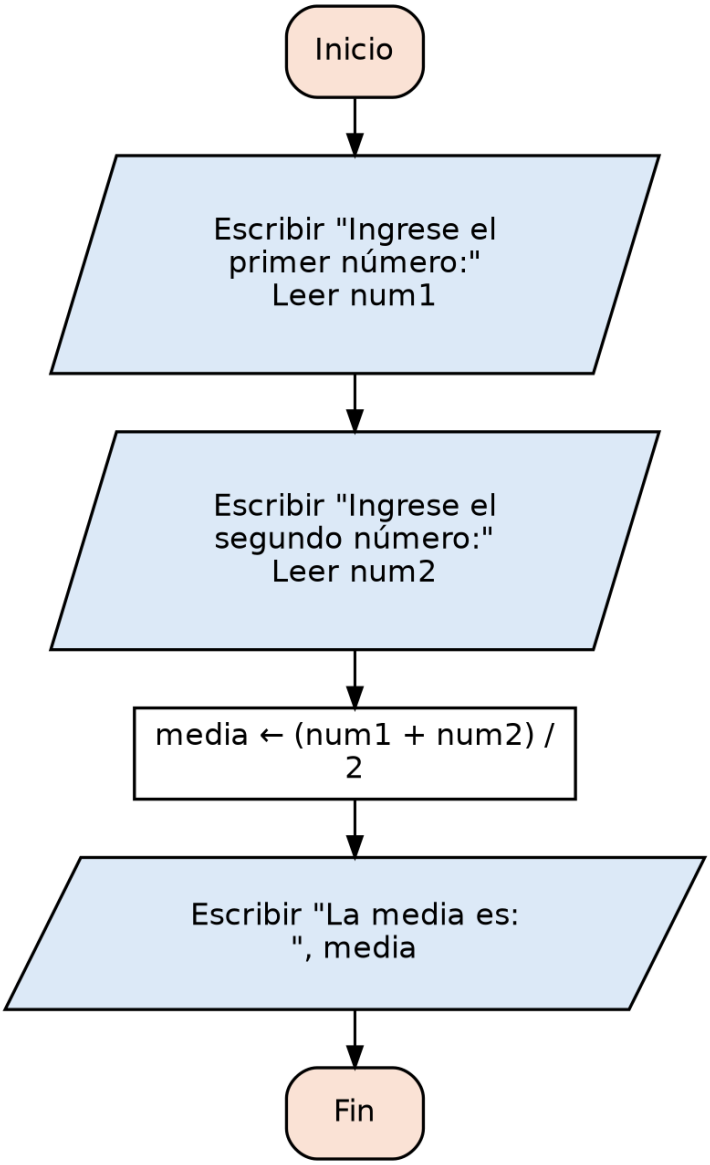
        // Proceso
        media <- (num1 + num2) / 2

        // Salida de datos
        Escribir "La media aritmetica es: ", media
    FinAlgoritmo
```

Prueba de escritorio

Paso	num1	num2	media	Acción / Condición
1	8	-	-	Se ingresa el valor de num1
2	8	12	-	Se ingresa el valor de num2
3	8	12	10.0	Se calcula (8 + 12) / 2

Diagrama de flujo



Código fuente

```
#include <stdio.h>

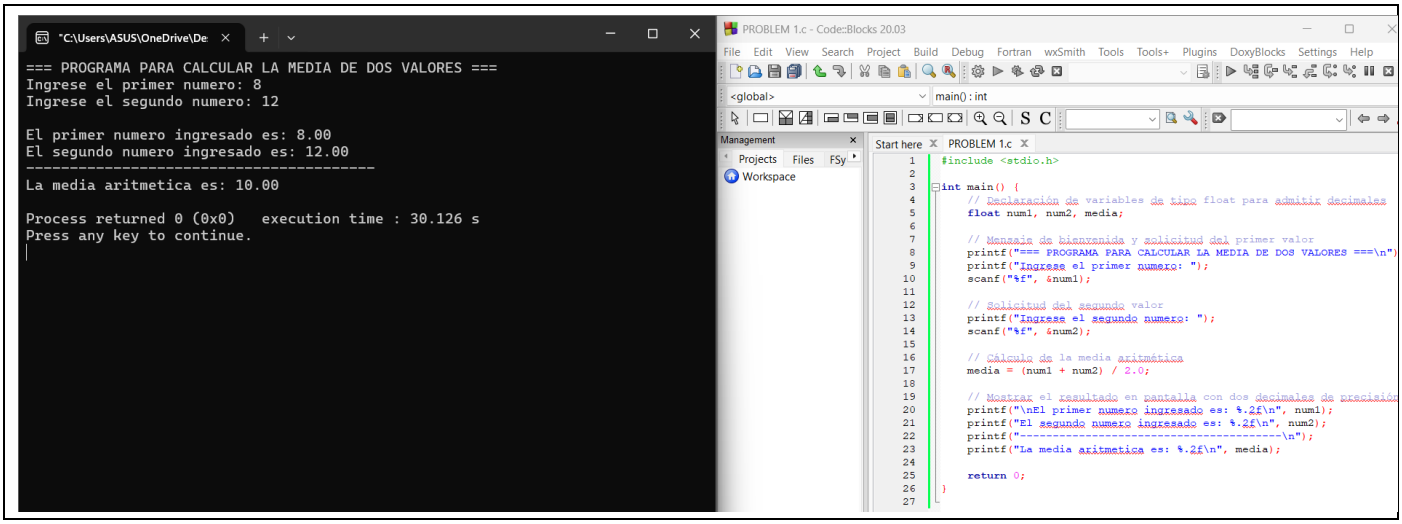
int main()
{
    double num1, num2, media;

    printf("Ingrese el primer numero: ");
    scanf("%lf", &num1);
    printf("Ingrese el segundo numero: ");
    scanf("%lf", &num2);

    media = (num1 + num2) / 2;

    printf("La media es: %.2lf\n", media);
    return 0;
}
```

Pantallazo de Code::Blocks



2- Valor absoluto de x al cubo

Desarrolla un programa que lea un número por teclado y escriba por pantalla $|x|^3$.

Análisis del problema

- **Entrada:** Un número x (entero o real) ingresado por teclado.
- **Proceso:** Calcular el valor absoluto de x (si es negativo, se multiplica por -1) y elevar ese resultado al cubo ($|x|^3$).
- **Salida:** El valor resultante del cubo del valor absoluto de x.
- **Restricciones:** El valor debe estar dentro del rango permitido por el tipo de dato para evitar desbordamientos al elevar al cubo.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	X	Ingresado por teclado	Real
Variable auxiliar	abs_x	Calculado	Real
Dato de salida	resultado	Calculado	Real

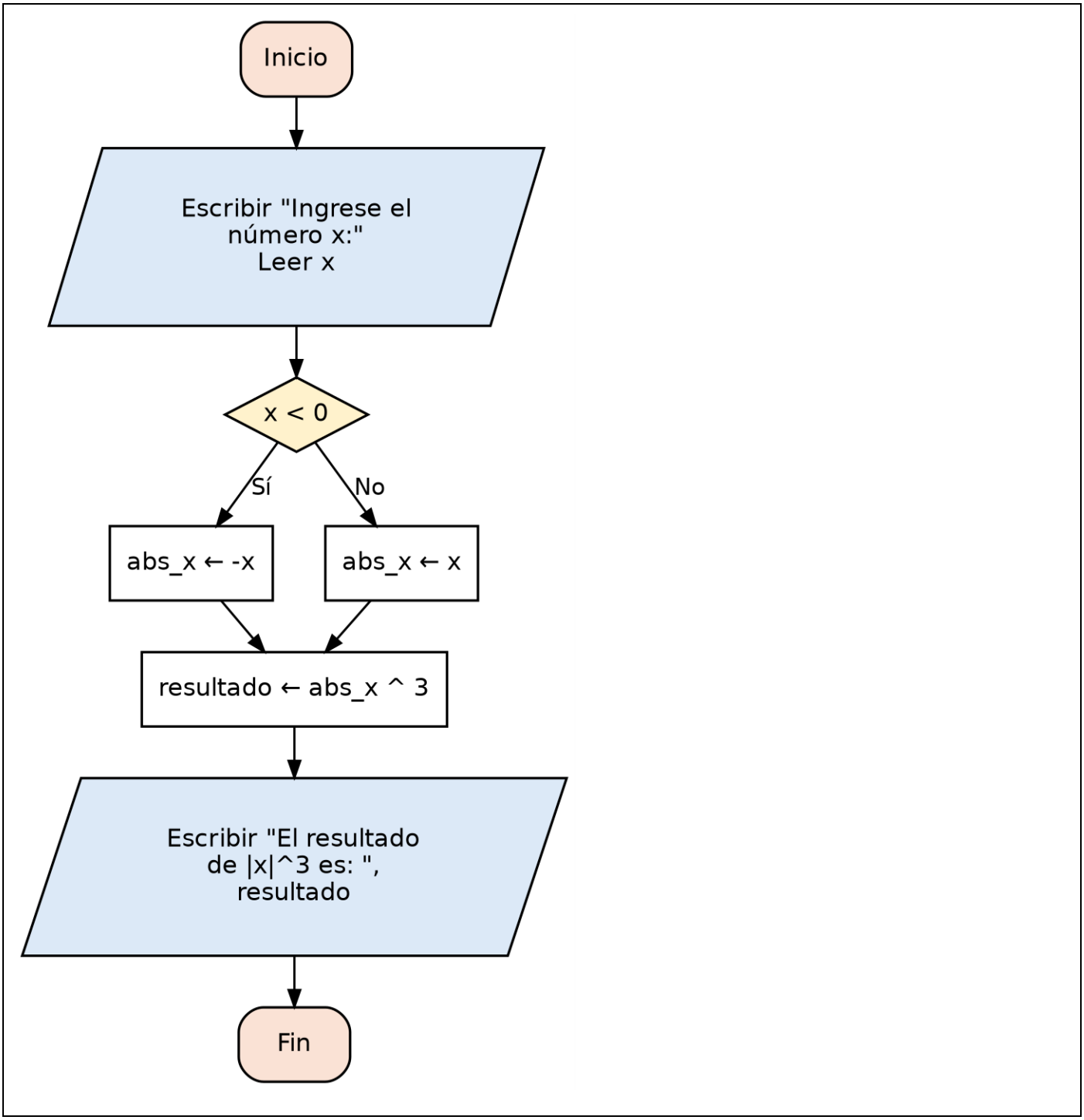
Seudocódigo

```
Proceso ValorAbsolutoCubo
   Definir x, abs_x, resultado Como Real
   Escribir "Ingrese el número x:"
   Leer x
   Si x < 0 Entonces
      abs_x <- -x
   Sino
      abs_x <- x
   FinSi
   resultado <- abs_x ^ 3
   Escribir "El resultado de |x|^3 es: ", resultado
FinProceso
```

Prueba de escritorio

Paso	x	abs_x	resultado	Acción / Condición
1	-3	-	-	Se ingresa x
2	-3	3	-	Como -3 < 0, abs_x = -(-3) = 3
3	-3	3	27.0	Se calcula 3 ^ 3 = 27

Diagrama de flujo



Código fuente

```c
#include <stdio.h>

int main()
{
    double x, abs_x, resultado;

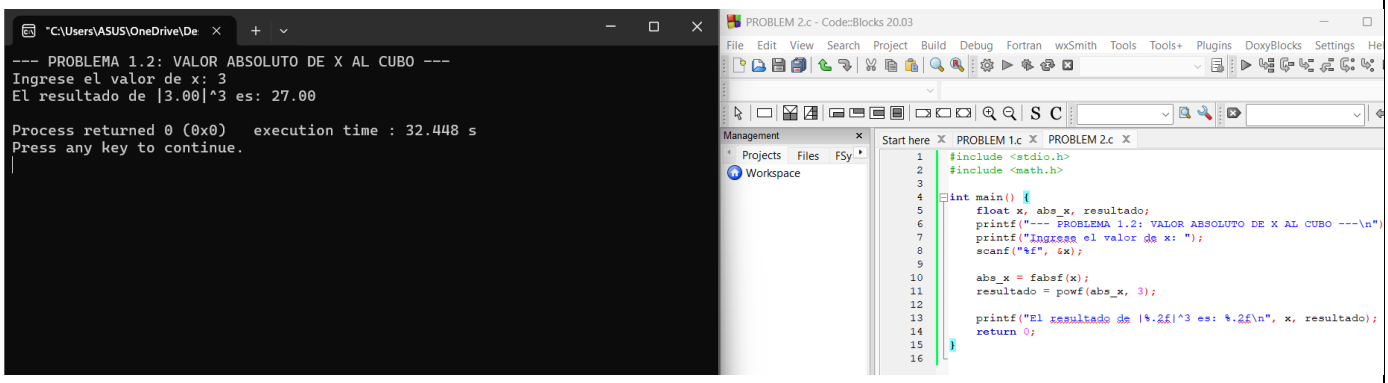
    printf("Ingrese el numero x: ");
    scanf("%lf", &x);

    if (x < 0)
    {
        abs_x = -x;
    }
    else
    {
        abs_x = x;
    }

    resultado = abs_x * abs_x * abs_x;

    printf("El resultado de |x|^3 es: %.2lf\n", resultado);
    return 0;
}
```

Pantallazo de Code::Blocks



3- Divisible

Desarrolla un programa que lea dos números enteros por teclado y determine si el primero es divisible por el segundo. Se mostrará por pantalla el resultado.

Pista: La solución utiliza el operador módulo (%), que devuelve el resto de la división entera entre números enteros. Si x es divisible entre y, el resto debe ser 0.

Análisis del problema

- **Entrada:** Dos números enteros (x e y) ingresados por teclado.
- **Proceso:** Verificar si el resto de la división entera del primer número entre el segundo es igual a cero usando el operador módulo (x \% y == 0).
- **Salida:** Un mensaje indicando si el primer número es divisible por el segundo o no.
- **Restricciones:** El segundo número (y) no puede ser cero (evitar división por cero).

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	Num1	Ingresado por teclado	Entero
Dato de entrada	Num2	Ingresado por teclado	Entero
Dato de salida	Es_divisible	Calculo	Lógico

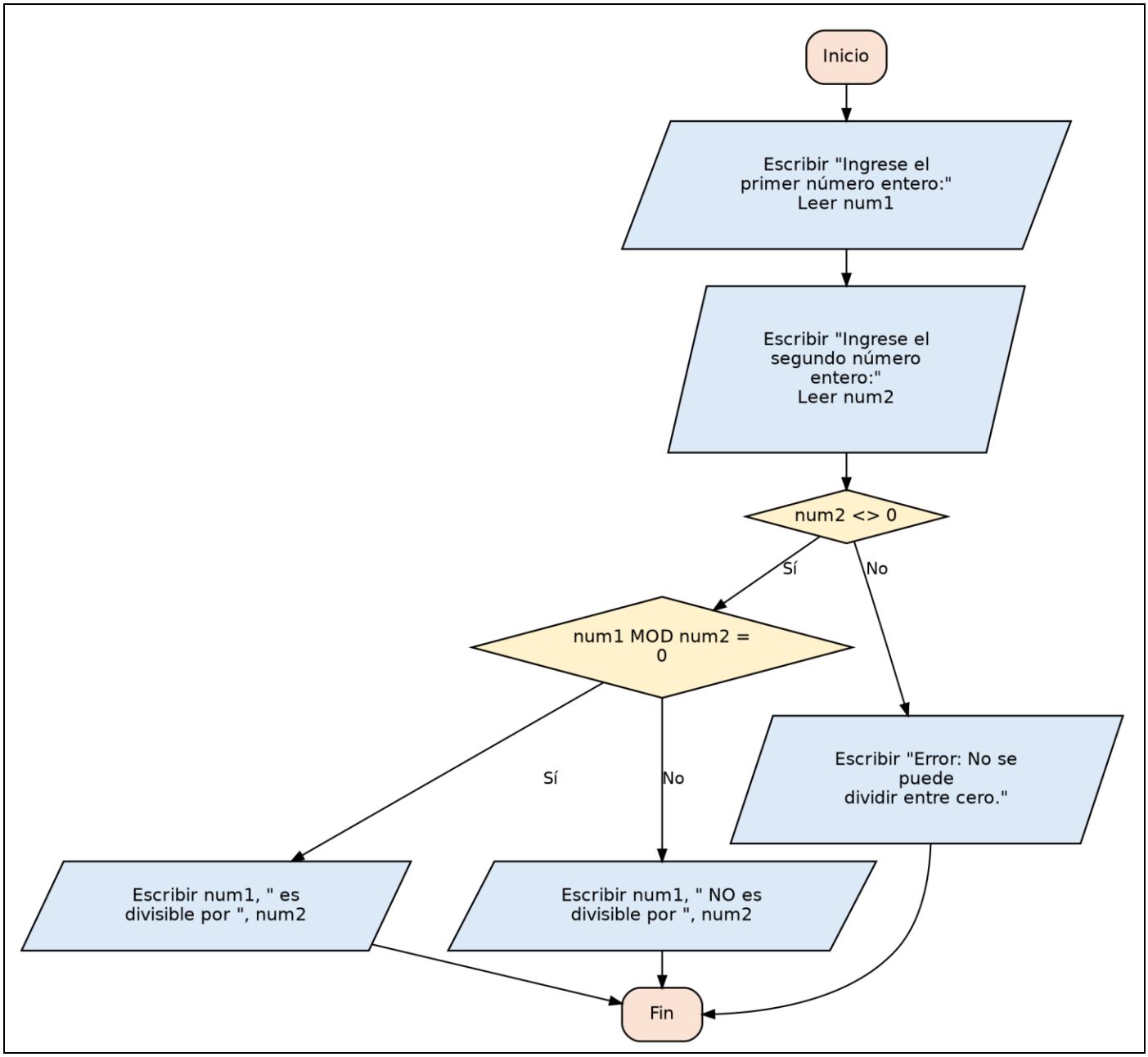
Seudocódigo

```
Proceso EsDivisible
   Definir num1, num2 Como Entero
   Escribir "Ingrese el primer número entero:"
   Leer num1
   Escribir "Ingrese el segundo número entero:"
   Leer num2
   Si num2 <> 0 Entonces
      Si num1 MOD num2 = 0 Entonces
         Escribir num1, " es divisible por ", num2
      Sino
         Escribir num1, " NO es divisible por ", num2
      FinSi
   Sino
      Escribir "Error: No se puede dividir entre cero."
   FinSi
FinProceso
```

Prueba de escritorio

Paso	num1	num2	Condición (num2 <> 0 y num1 MOD num2 = 0)	Salida / Mensaje
1	20	4	Verdadero (4 <> 0 y 20 MOD 4 = 0)	"20 es divisible por 4"

Diagrama de flujo



Código fuente

```
#include <stdio.h>

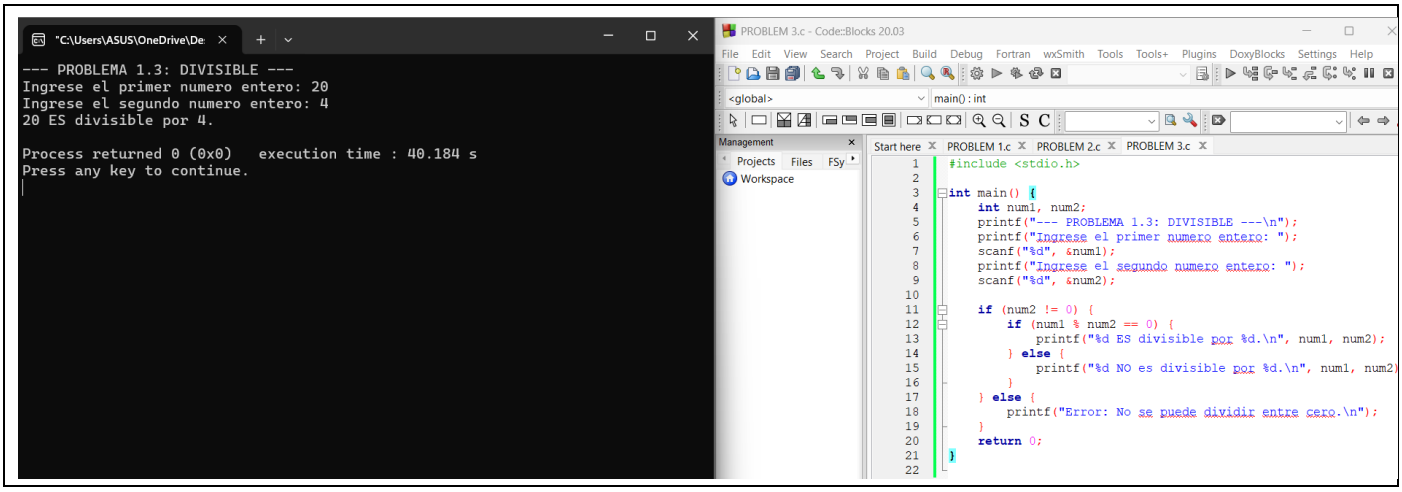
int main()
{
    int num1, num2;

    printf("Ingrese el primer numero entero: ");
    scanf("%d", &num1);
    printf("Ingrese el segundo numero entero: ");
    scanf("%d", &num2);

    if (num2 != 0)
    {
```

```
        if (num1 % num2 == 0)
        {
            printf("%d es divisible por %d\n", num1, num2);
        }
        else
        {
            printf("%d NO es divisible por %d\n", num1, num2);
        }
    }
    else
    {
        printf("Error: No se puede dividir entre cero.\n");
    }
    return 0;
}
```

Pantallazo de Code::Blocks



4- Intervalo

Desarrolla un programa que lea un número por teclado y determine si pertenece al intervalo [0, 10], indicando el resultado por pantalla.

Pista: El número introducido debe cumplir x ≥ 0 y x ≤ 10 (ambas condiciones a la vez) para pertenecer al intervalo. Para establecer estas dos condiciones se utiliza un operador lógico: el operador Y en el diagrama de flujo y && en C. Evita la tentación de escribir 0 ≤ x ≤ 10 como una sola condición, porque en C no se evalúa como esperas.

Análisis del problema

- **Entrada:** Un número x (entero o real) introducido por teclado.
- **Proceso:** Evaluar simultáneamente mediante un operador lógico AND (&&) si el número cumple que x ≥ 0 y x ≤ 10.
- **Salida:** Un mensaje indicando si el número pertenece o no al intervalo [0, 10].
- **Restricciones:** ninguna.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	X	Ingresado por teclado	Real
Dato de salida	En intervalo	calculado	Lógico

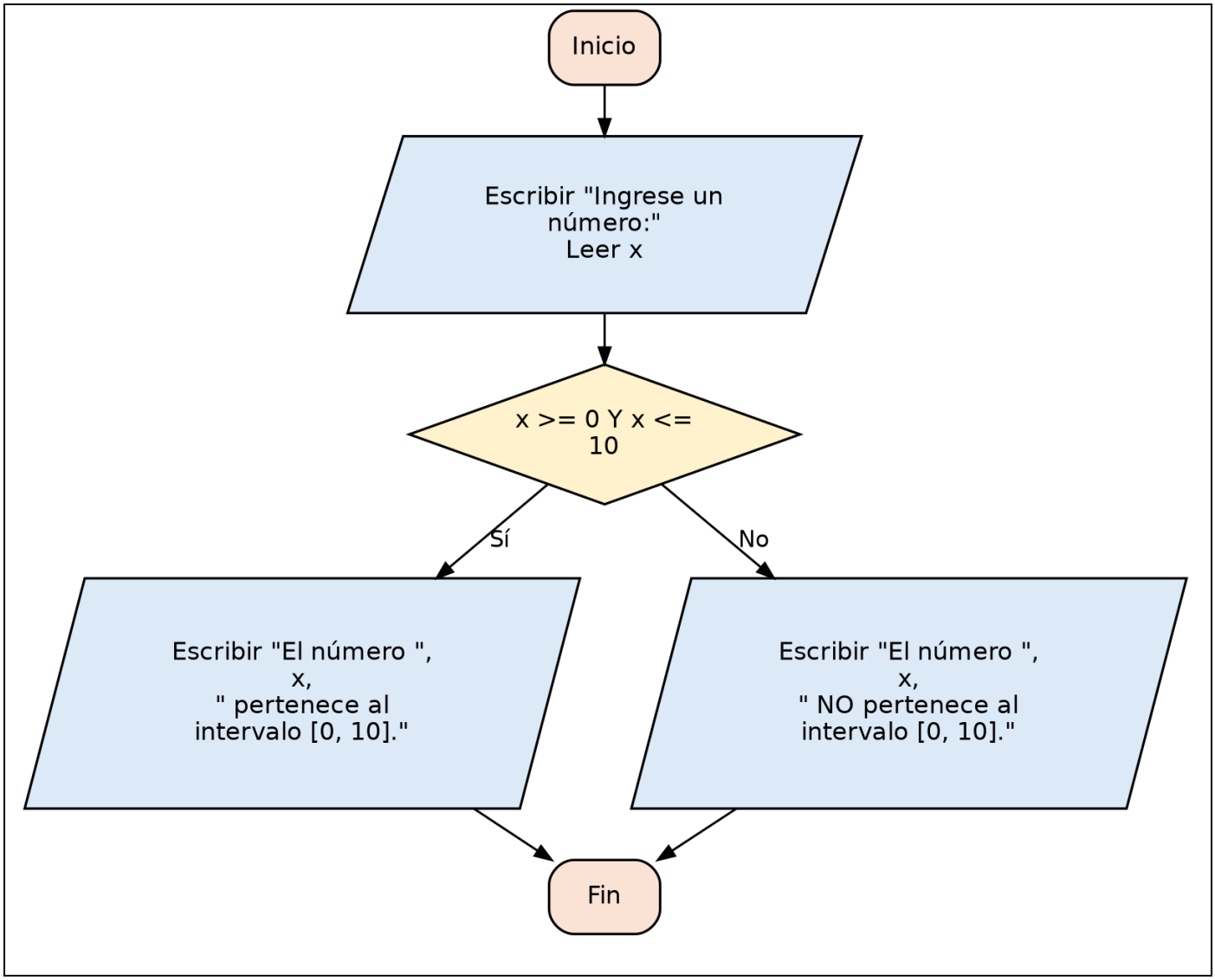
Seudocódigo

```
Proceso VerificarIntervalo
  Definir x Como Real
  Escribir "Ingrese un número:"
  Leer x
  Si x >= 0 Y x <= 10 Entonces
    Escribir "El número ", x, " pertenece al intervalo [0, 10]."
  Sino
    Escribir "El número ", x, " NO pertenece al intervalo [0, 10]."
  FinSi
FinProceso
```

Prueba de escritorio

Paso	x	Condición (x >= 0 Y x <= 10)	Salida / Mensaje
1	7.5	Verdadero (7.5 >= 0 y 7.5 <= 10)	"El número 7.5 pertenece al intervalo [0, 10]."

Diagrama de flujo



Código fuente

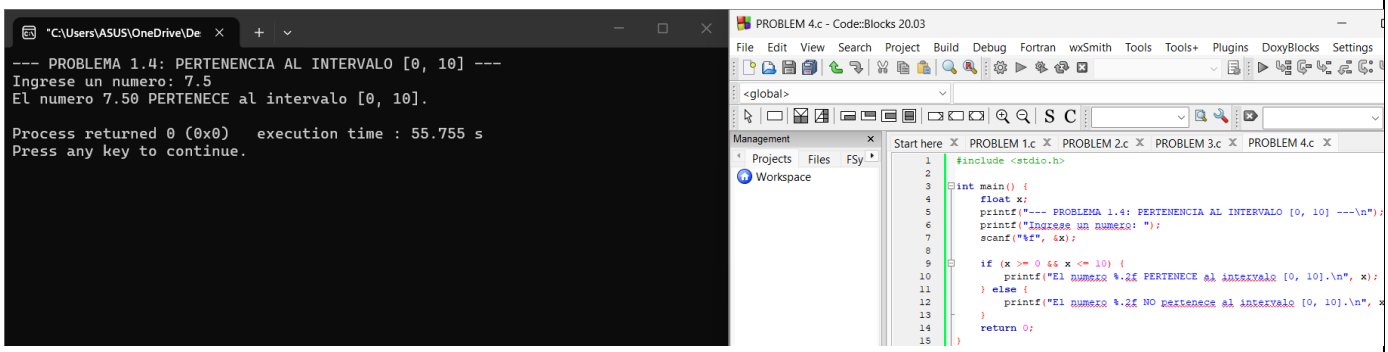
```
#include <stdio.h>

int main()
{
    double x;

    printf("Ingrese un numero: ");
    scanf("%lf", &x);

    if (x >= 0 && x <= 10)
    {
        printf("El numero %.2lf pertenece al intervalo [0, 10].\n", x);
    }
    else
    {
        printf("El numero %.2lf NO pertenece al intervalo [0, 10].\n", x);
    }
    return 0;
}
```

Pantallazo de Code::Blocks



5- Conversión de unidades de tiempo

Desarrolla un programa que lea por teclado un valor entero c, interpretándolo como el número de segundos que dura un evento, y calcule y muestre por pantalla cuántos días, horas, minutos y segundos representa.

Para calcular los días, horas, minutos y segundos se utilizan la división entera y el módulo (resto de una división, operador %). Sea c el número de segundos a convertir. Como un día tiene 24 horas, una hora tiene 60 minutos y un minuto tiene 60 segundos, el cociente entero c / (24 · 60 · 60) es el número de días completos contenidos en c. Asimismo, c % (24 · 60 · 60) es el número de segundos restantes tras descontar esos días. Se procede de igual manera para obtener las horas (1 hora = 60 · 60 segundos) y los minutos (1 minuto = 60 segundos).

Ejemplo: supón que quieres convertir c = 10 000 segundos. Como 10 000 / (24 · 60 · 60) = 0, hay 0 días, porque 10 000 segundos es menos que un día. Análogamente, 10 000 / (60 · 60) = 2 horas. Si a 10 000 segundos les quitas esas 2 horas, quedan 10 000 % (60 · 60) = 2 800 segundos, que son algo más de 46 minutos, cantidad que se obtiene con 2 800 / 60 = 46 minutos. Finalmente, 2 800 % 60 = 40 segundos. Por lo tanto, 10 000 segundos equivalen a 0 días, 2 horas, 46 minutos y 40 segundos.

Análisis del problema

- **Entrada:** Un valor entero c que representa el número total de segundos.
- **Proceso:**

i- Calcular días dividiendo c entre 24*3600 y actualizar el residuo con módulo.
ii- Calcular horas dividiendo los segundos restantes entre 3600 y actualizar el residuo.
iii- Calcular minutos dividiendo entre 60 y actualizar el residuo para obtener los segundos finales.

- **Salida:** La equivalencia del tiempo en días, horas, minutos y segundos.
- **Restricciones:** El valor de c debe ser un número entero mayor o igual a 0.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	C	Ingresado por teclado	Entero
Variable auxiliar	Resto_segundos	calculado	Entero
Dato de salida	Días	Calculado	Entero
Dato de salida	horas	calculado	Entero
Dato de salida	minutos	calculado	Entero
Sato de salida	segundos	calculado	Entero

Seudocódigo

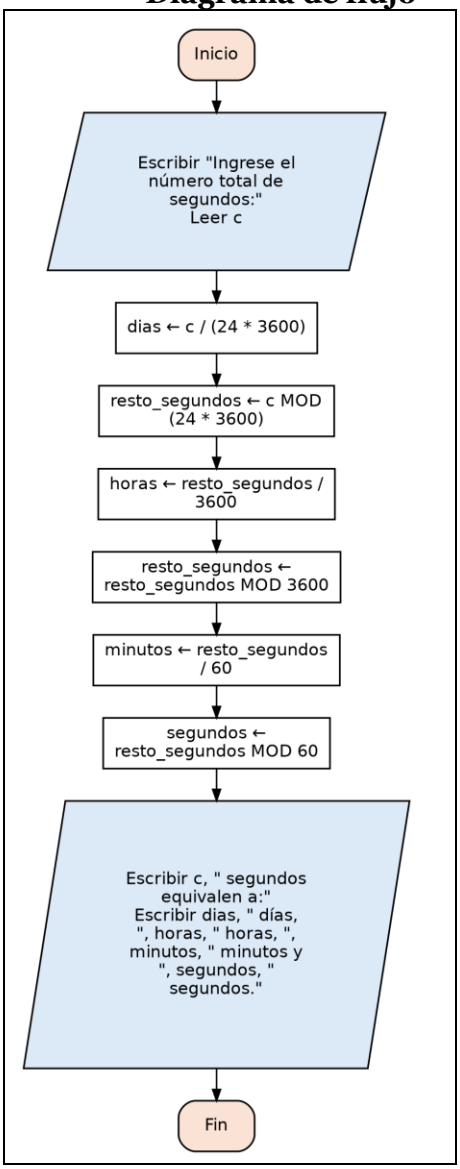
```
Proceso ConvertirTiempo
   Definir c, resto_segundos, dias, horas, minutos, segundos Como Entero
   Escribir "Ingrese el número total de segundos:"
   Leer c
   dias <- c / (24 * 3600)
   resto_segundos <- c MOD (24 * 3600)
   horas <- resto_segundos / 3600
   resto_segundos <- resto_segundos MOD 3600
   minutos <- resto_segundos / 60
   segundos <- resto_segundos MOD 60
   Escribir c, " segundos equivalen a:"
   Escribir dias, " días, ", horas, " horas, ", minutos, " minutos y ", segundos, " segundos."
FinProceso
```

Prueba de escritorio

Paso	c	resto_segundos	dias	horas	minutos	segundos
1	10000	-	-	-	-	-

2	10000	10000	0	-	-	-
3	10000	2800	0	2	-	-
4	10000	40	0	2	46	-
5	10000	40	0	0	0	40

Diagrama de flujo



Código fuente

```
#include <stdio.h>

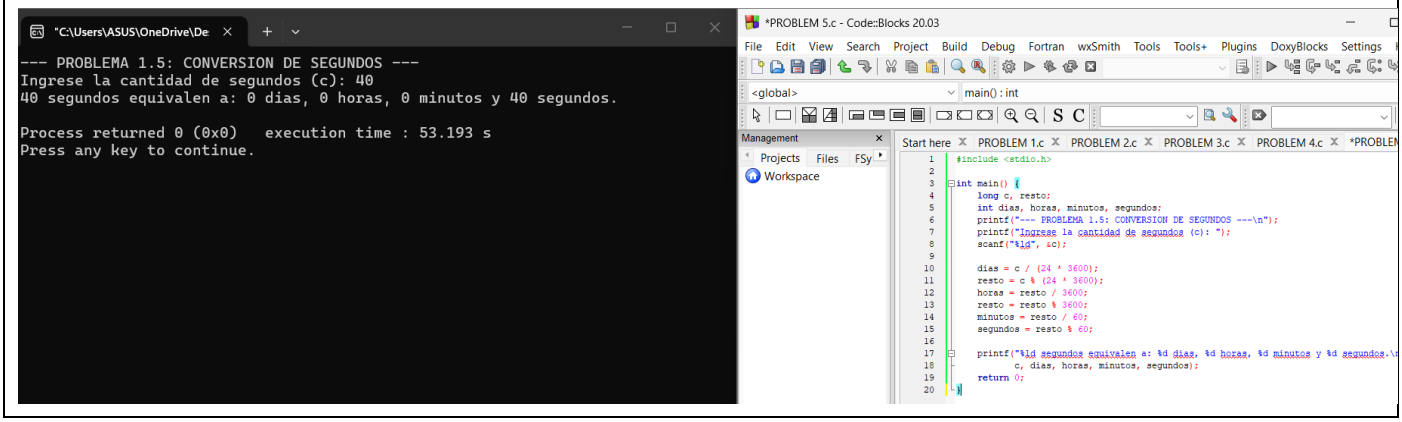
int main()
{
    int c, resto_segundos, dias, horas, minutos, segundos;

    printf("Ingrese el numero total de segundos: ");
    scanf("%d", &c);

    dias = c / (24 * 3600);
    resto_segundos = c % (24 * 3600);
    horas = resto_segundos / 3600;
    resto_segundos = resto_segundos % 3600;
    minutos = resto_segundos / 60;
    segundos = resto_segundos % 60;

    printf("%d segundos equivalen a:\n", c);
    printf("%d dias, %d horas, %d minutos y %d segundos.\n", dias, horas, minutos, segundos);
    return 0;
}
```

Pantallazo de Code::Blocks



6- Máximo, mínimo y media de alturas

Desarrolla un programa que lea por teclado un número indeterminado de alturas y deje de leer cuando se introduzca una negativa. El programa debe calcular e imprimir la media, el máximo y el mínimo de las alturas introducidas.

Análisis del problema

- **Entrada:** Una secuencia indeterminada de números reales que representan alturas.
- **Proceso:** Leer números en un bucle acumulando la suma, contando los elementos y actualizando los valores máximo y mínimo en cada iteración hasta recibir un número negativo.
- **Salida:** La media, el valor máximo y el valor mínimo de las alturas introducidas.
- **Restricciones:** El primer número negativo indica el fin de la lectura y no participa en los cálculos. Se debe ingresar al menos una altura válida.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	altura	Ingresado por teclado	Real
Variable auxiliar	Suma	0.0	Real
Variable auxiliar	contador	0	Entero
Dato de salida	Máximo	Evaluado	Real
Dato de salida	Mínimo	Evaluado	Real
Dato de salida	media	Calculado	Real

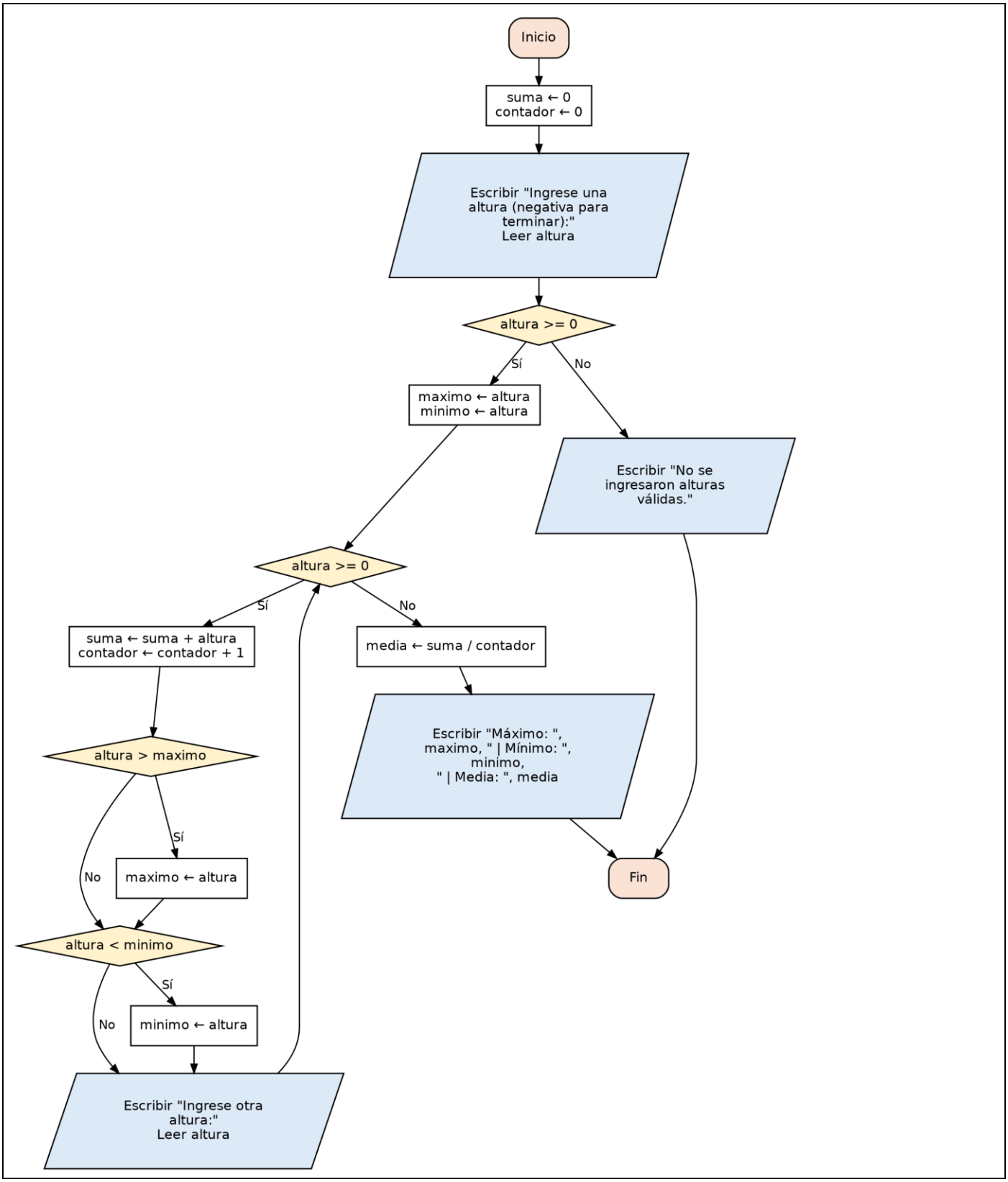
Seudocódigo

```
Proceso AlturasEstadisticas
  Definir altura, suma, maximo, minimo, media Como Real
  Definir contador Como Entero
  suma <- 0
  contador <- 0
  Escribir "Ingrese una altura (negativa para terminar):"
  Leer altura
  Si altura >= 0 Entonces
    maximo <- altura
    minimo <- altura
    Mientras altura >= 0 Hacer
      suma <- suma + altura
      contador <- contador + 1
      Si altura > maximo Entonces
        maximo <- altura
      FinSi
      Si altura < minimo Entonces
        minimo <- altura
      FinSi
      Escribir "Ingrese otra altura:"
      Leer altura
    FinMientras
    Si contador > 0 Entonces
      media <- suma / contador
      Escribir "Altura máxima: ", maximo
      Escribir "Altura mínima: ", minimo
      Escribir "Media de alturas: ", media
    FinSi
  Sino
    Escribir "No se ingresaron alturas válidas."
  FinSi
FinProceso
```

Prueba de escritorio

Iteración	altura	suma	contador	maximo	minimo
Inicial	1.70	0.0	0	-	-
1	1.70	1.70	1	1.70	1.70
2	1.85	3.55	2	1.85	1.70
3	1.60	5.15	3	1.85	1.60
4 (Fin)	-1	5.15	3	1.85	1.60 (media = 5.15/3 = 1.716)

Diagrama de flujo



Código fuente

```
#include <stdio.h>

int main()
{
    double altura, suma = 0, maximo, minimo, media;
    int contador = 0;

    printf("Ingrese una altura (negativa para terminar): ");
    scanf("%lf", &altura);

    if (altura >= 0)
    {
        maximo = altura;
        minimo = altura;

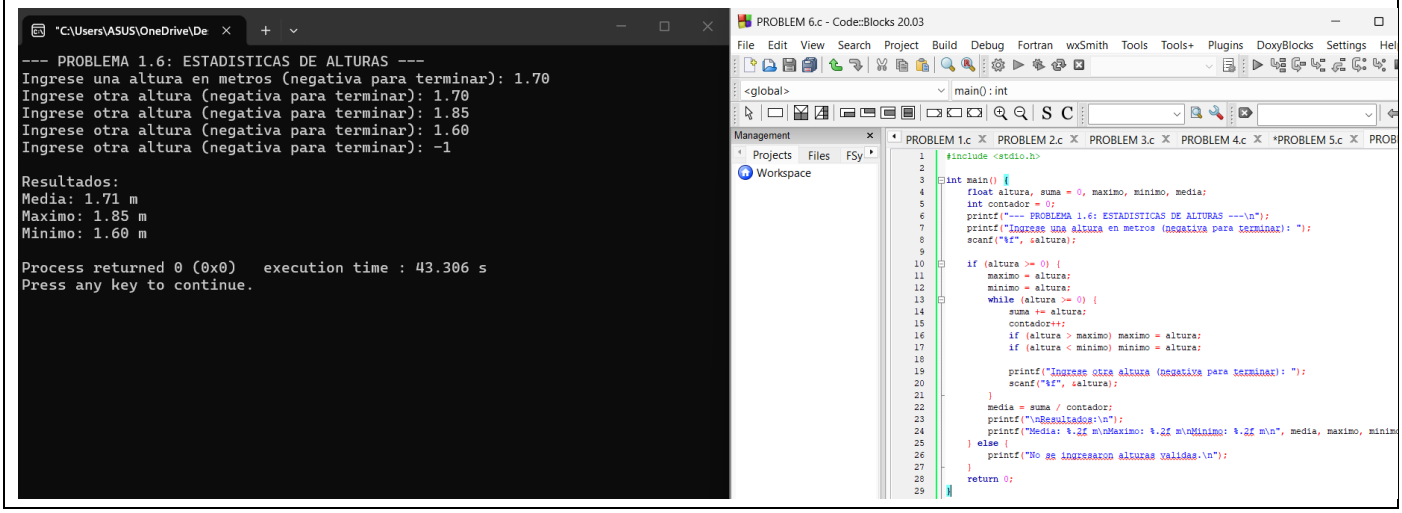
        while (altura >= 0)
        {
            suma = suma + altura;
            contador = contador + 1;

            if (altura > maximo)
            {
                maximo = altura;
            }
            if (altura < minimo)
            {
                minimo = altura;
            }

            printf("Ingrese otra altura: ");
            scanf("%lf", &altura);
        }

        media = suma / contador;
        printf("Maximo: %.2lf | Minimo: %.2lf | Media: %.2lf\n", maximo, minimo, media);
    }
    else
    {
        printf("No se ingresaron alturas validas.\n");
    }
    return 0;
}
```

Pantallazo de Code::Blocks



7- Raíz cuadrada entera

Desarrolla un programa que calcule la raíz cuadrada entera por defecto de un número N positivo dado, que se define como el mayor número entero R > 0 tal que $R^2 \leq N$.

Pista: Dada una variable entera i = 1, itera incrementando su valor de uno en uno mientras $i^2 \leq N$. La aproximación por defecto de la raíz es R = i – 1.

Análisis del problema

- **Entrada:** Un número entero positivo N.
- **Proceso:** Utilizar una variable iterativa i que comience en 1 y se incremente de uno en uno mientras $i^2 \leq N$. La aproximación por defecto será R = i - 1.
- **Salida:** La raíz cuadrada entera por defecto (R).
- **Restricciones:** N debe ser un número entero estrictamente positivo (N > 0).

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	N	Ingresado por teclado	Entero
Variable auxiliar	i	1	Entero
Dato de salida	R	Calculado	Entero

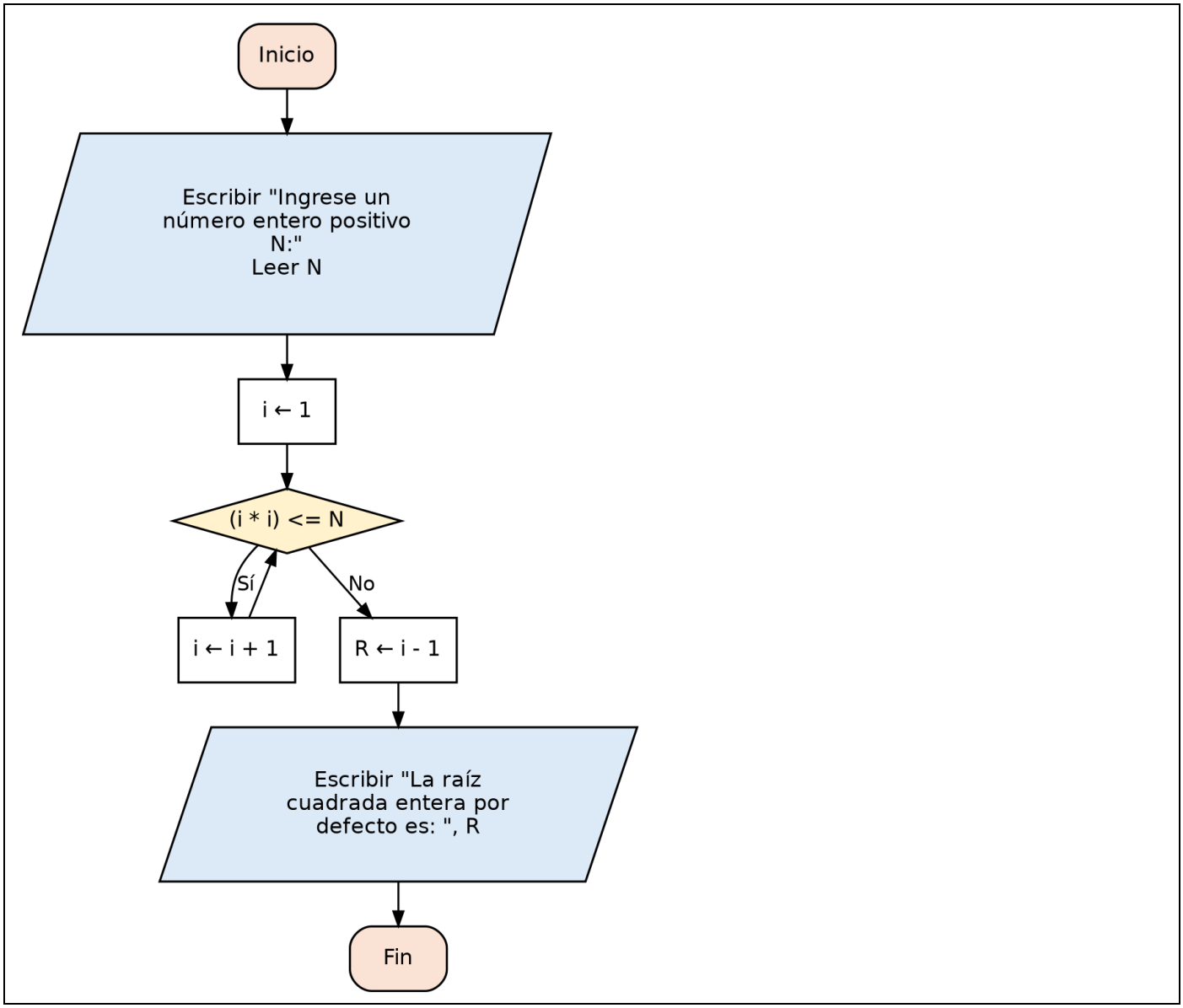
Seudocódigo

```
Proceso RaizCuadradaEntera
   Definir N, i, R Como Entero
   Escribir "Ingrese un número entero positivo N:"
   Leer N
   i <- 1
   Mientras (i * i) <= N Hacer
      i <- i + 1
   FinMientras
   R <- i - 1
   Escribir "La raíz cuadrada entera por defecto es: ", R
FinProceso
```

Prueba de escritorio

Iteración	N	i	Condición (i * i) <= N
Inicial	20	1	Verdadero (1 <= 20)
1	20	2	Verdadero (4 <= 20)
2	20	3	Verdadero (9 <= 20)
3	20	4	Verdadero (16 <= 20)
4	20	5	Falso (25 <= 20 -> R = 5 - 1 = 4)

Diagrama de flujo



Código fuente

```
#include <stdio.h>

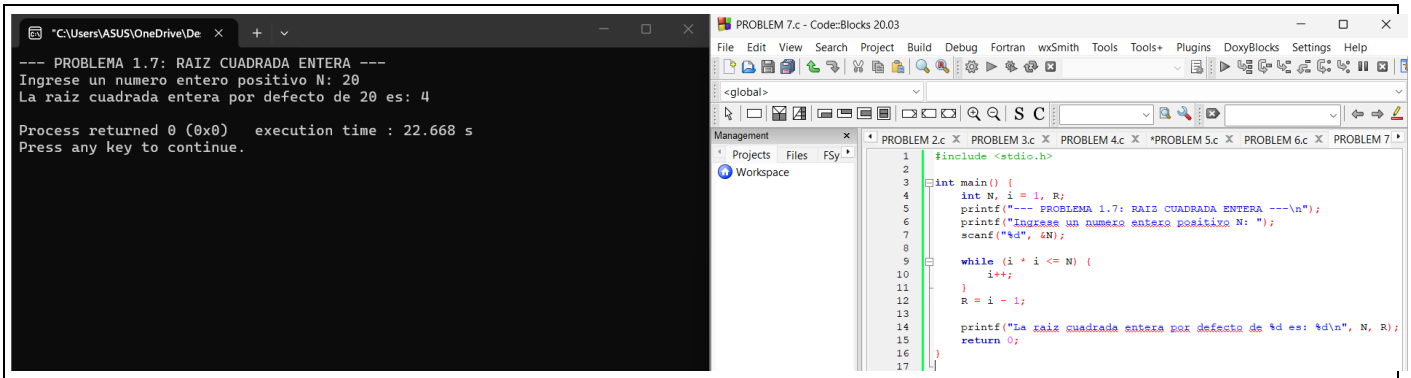
int main()
{
    int N, i, R;

    printf("Ingrese un numero entero positivo N: ");
    scanf("%d", &N);

    i = 1;
    while (i * i <= N)
    {
        i = i + 1;
    }
    R = i - 1;
```

```
    printf("La raiz cuadrada entera por defecto es: %d\n", R);
    return 0;
}
```

Pantallazo de Code::Blocks



8- Número primo

Desarrolla un programa que ingrese por teclado un número entero mayor que 1 y determine si es primo o no.

Análisis del problema

- **Entrada:** Un número entero introducido por teclado.
- **Proceso:** Comprobar si el número posee divisores exactos distintos de 1 y de sí mismo, evaluando los posibles divisores en un rango adecuado.
- **Salida:** Un mensaje indicando si el número es primo o no.
- **Restricciones:** El número ingresado debe ser mayor que 1.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	num	Ingresado por teclado	entero
Variable auxiliar	i	2	Entero
Variable auxiliar	es_primo	Verdadero (1)	Lógico
Dato de salida	resultado	Evaluado	Cadena de texto

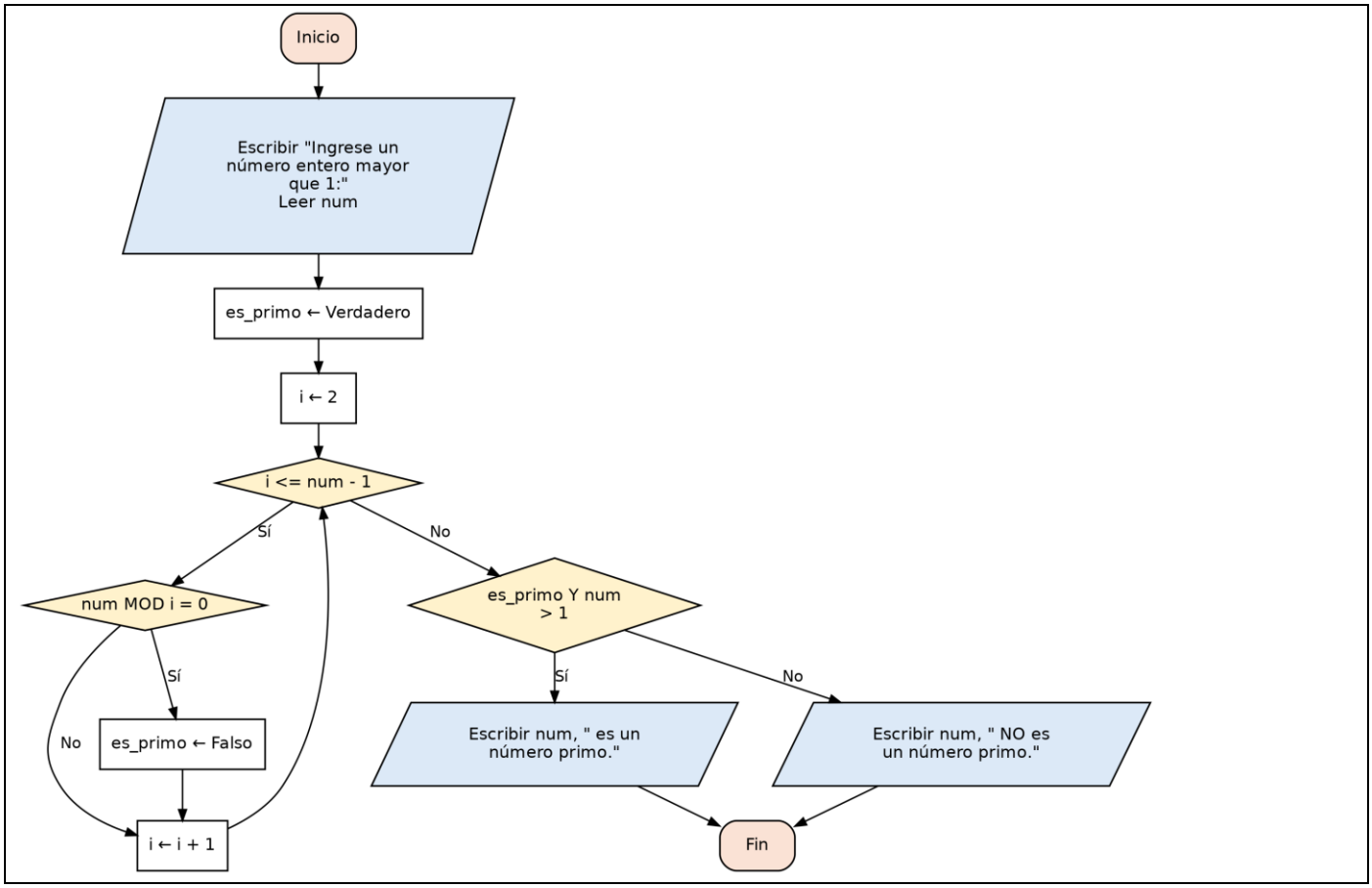
Seudocódigo

```
Proceso VerificarPrimo
   Definir num, i Como Entero
   Definir es_primo Como Logico
   Escribir "Ingrese un número entero mayor que 1:"
   Leer num
   es_primo <- Verdadero
   Para i <- 2 Hasta num - 1 Hacer
      Si num MOD i = 0 Entonces
         es_primo <- Falso
      FinSi
   FinPara
   Si es_primo Y num > 1 Entonces
      Escribir num, " es un número primo."
   Sino
      Escribir num, " NO es un número primo."
   FinSi
FinProceso
```

Prueba de escritorio

i	num MOD i = 0?	es_primo
2	7 MOD 2 = 1 (Falso)	Verdadero
3	7 MOD 3 = 1 (Falso)	Verdadero
4	7 MOD 4 = 3 (Falso)	Verdadero
5	7 MOD 5 = 2 (Falso)	Verdadero
6	7 MOD 6 = 1 (Falso)	Verdadero (Salida: 7 es primo)

Diagrama de flujo



Código fuente

```
#include <stdio.h>

int main()
{
    int num, i, es_primo;

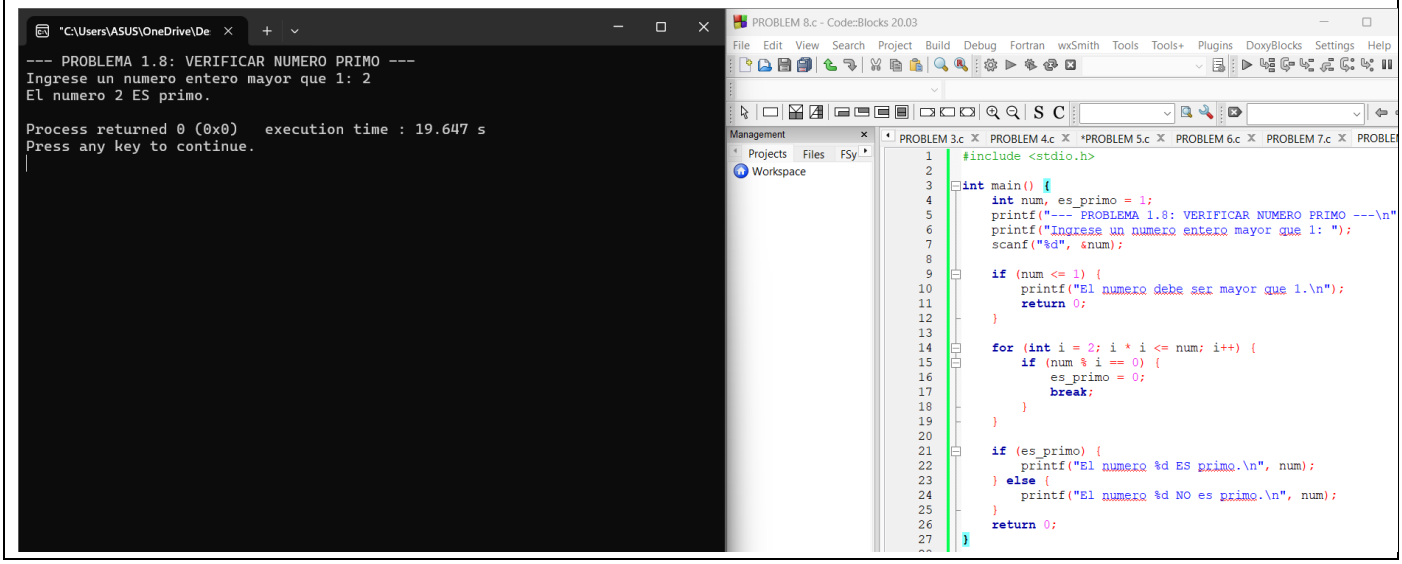
    printf("Ingrese un numero entero mayor que 1: ");
    scanf("%d", &num);

    es_primo = 1;
    for (i = 2; i <= num - 1; i++)
    {
        if (num % i == 0)
        {
            es_primo = 0;
        }
    }

    if (es_primo && num > 1)
    {
        printf("%d es un numero primo.\n", num);
    }
    else
    {
        printf("%d NO es un numero primo.\n", num);
    }
```

```
    return 0;
}
```

Pantallazo de Code::Blocks



9- Números primos comprendidos entre dos números dados

Desarrolla un programa que muestre por pantalla todos los números primos comprendidos entre dos números enteros introducidos por teclado.

Análisis del problema

- **Entrada:** Dos números enteros que establecen un rango o límites.
- **Proceso:** Recorrer secuencialmente todos los números enteros comprendidos entre los dos límites y aplicar la lógica de primalidad a cada uno.
- **Salida:** La lista de todos los números primos encontrados dentro de ese rango.
- **Restricciones:** Manejar correctamente el orden de los límites en caso de que el límite inferior sea mayor que el superior.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	limite_inferior	Ingresado por teclado	Entero
Dato de entrada	limite_superior	Ingresado por teclado	Entero
Variable auxiliar	num_actual	Sin inicializar	Entero
Variable auxiliar	i	Sin inicializar	Entero
Variable auxiliar	es_primo	Sin inicializar	Lógico
Dato de salida	primo_encontrado	Impreso en pantalla	Entero

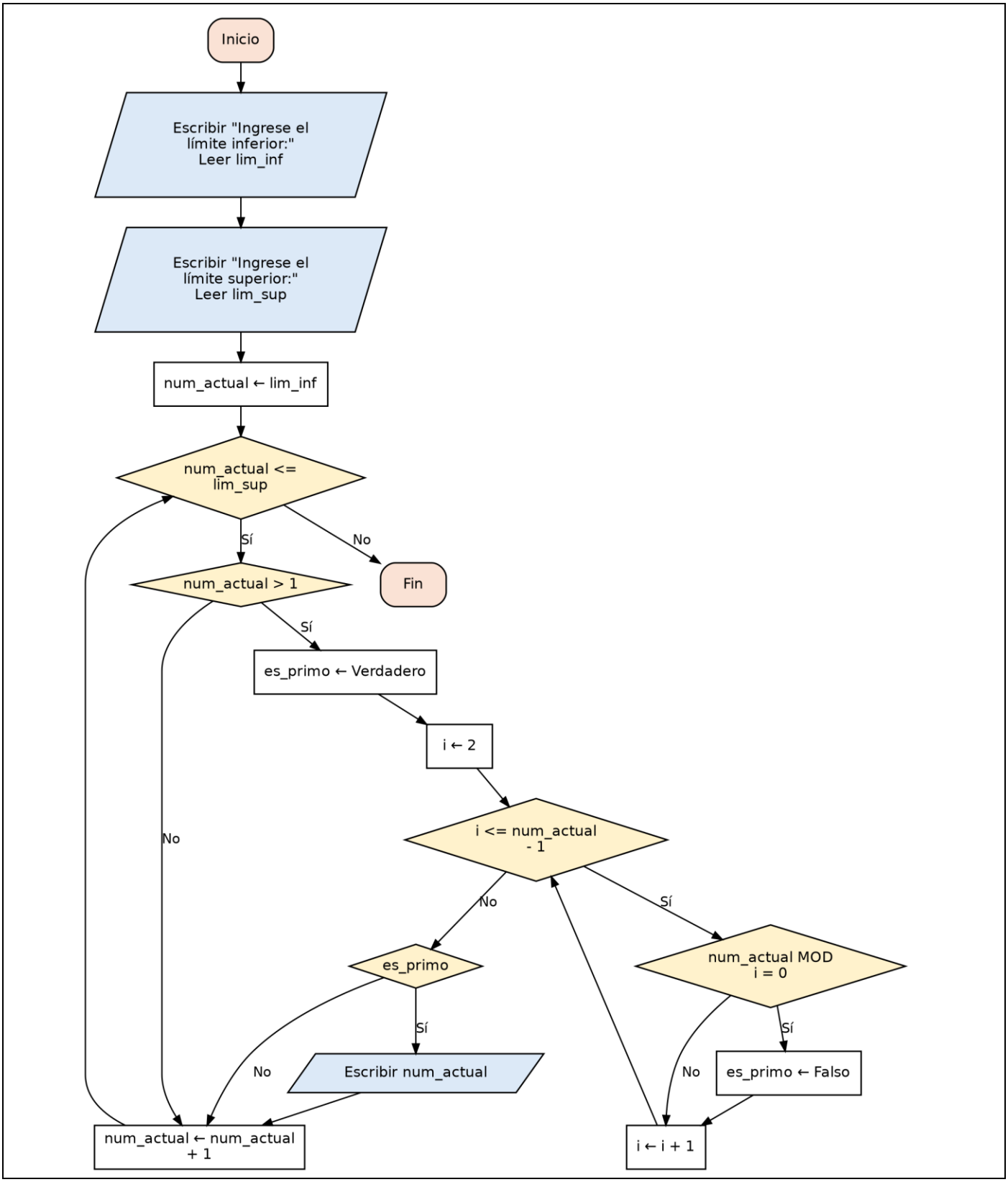
Seudocódigo

```
Proceso PrimosEnRango
   Definir lim_inf, lim_sup, num_actual, i Como Entero
   Definir es_primo Como Logico
   Escribir "Ingrese el límite inferior:"
   Leer lim_inf
   Escribir "Ingrese el límite superior:"
   Leer lim_sup
   Escribir "Números primos entre ", lim_inf, " y ", lim_sup, ":"
   Para num_actual <- lim_inf Hasta lim_sup Hacer
      Si num_actual > 1 Entonces
         es_primo <- Verdadero
         Para i <- 2 Hasta num_actual - 1 Hacer
            Si num_actual MOD i = 0 Entonces
               es_primo <- Falso
            FinSi
         FinPara
         Si es_primo Entonces
            Escribir num_actual
         FinSi
      FinSi
   FinPara
FinProceso
```

Prueba de escritorio

num_actual	Evaluación de primalidad	Acción
10	10 no es primo (divisible por 2 y 5)	Se omite
11	11 es primo	Se imprime 11
12	12 no es primo	Se omite
13	13 es primo	Se imprime 13
14	14 no es primo	Se omite
15	15 no es primo	Se omite

Diagrama de flujo



Código fuente

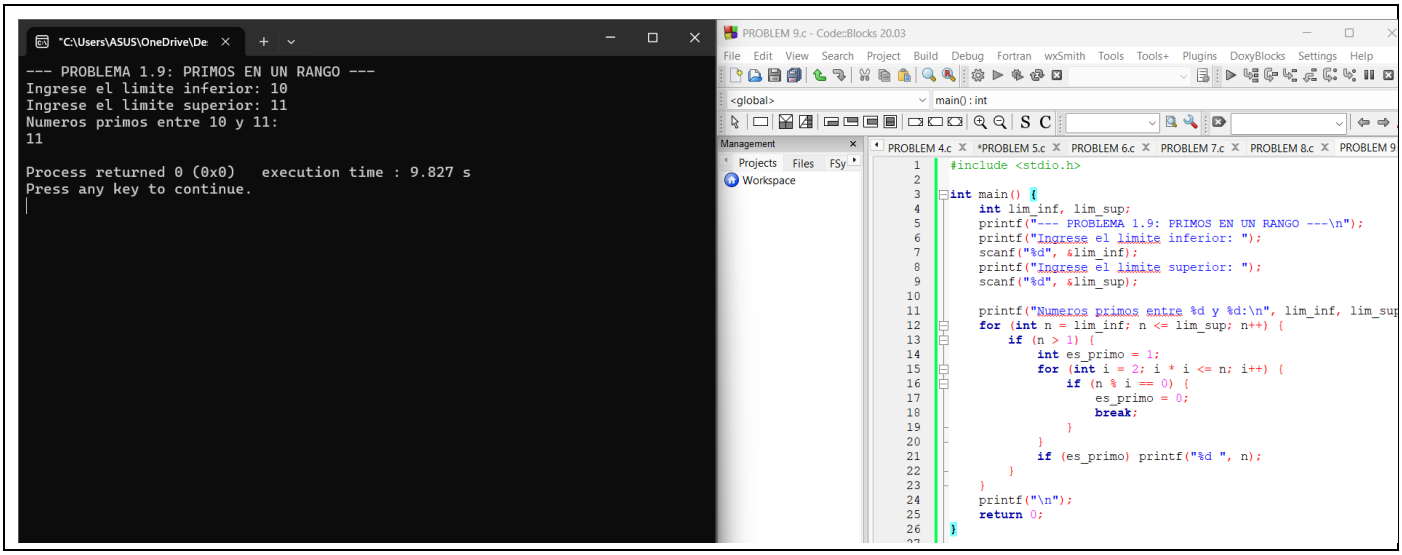
```c
#include <stdio.h>

int main()
{
    int lim_inf, lim_sup, num_actual, i, es_primo;

    printf("Ingrese el limite inferior: ");
    scanf("%d", &lim_inf);
    printf("Ingrese el limite superior: ");
    scanf("%d", &lim_sup);

    printf("Numeros primos entre %d y %d:\n", lim_inf, lim_sup);
    for (num_actual = lim_inf; num_actual <= lim_sup; num_actual++)
    {
        if (num_actual > 1)
        {
            es_primo = 1;
            for (i = 2; i <= num_actual - 1; i++)
            {
                if (num_actual % i == 0)
                {
                    es_primo = 0;
                }
            }
            if (es_primo)
            {
                printf("%d ", num_actual);
            }
        }
    }
    printf("\n");
    return 0;
}
```

Pantallazo de Code::Blocks



10-Factorial

Desarrolla un programa que lea un número entero n ≥ 0 y calcule n! = 1 · 2 · … · n, sabiendo que 0! = 1.

Ejemplo: n = 5 → 120.

Pista: Usa un acumulador que inicie en 1 y se multiplique en cada vuelta del ciclo.

Para pensar: ¿Qué ocurre si el usuario ingresa un número negativo? ¿Y con n = 20 en una variable de tipo int?

Análisis del problema

- **Entrada:** Un número entero n ≥ 0.
- **Proceso:** Utilizar un acumulador inicializado en 1 y un ciclo multiplicativo desde 1 hasta n.
- **Salida:** El valor del factorial n!.
- **Restricciones:** n ≥ 0. Tener en cuenta las limitaciones de desbordamiento (overflow) para valores grandes (por ejemplo, n=20 en tipos de datos estándar).

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	Ingresado por teclado	Entero
Variable auxiliar	i	1	Entero
Dato de salida	factorial	1	Entero

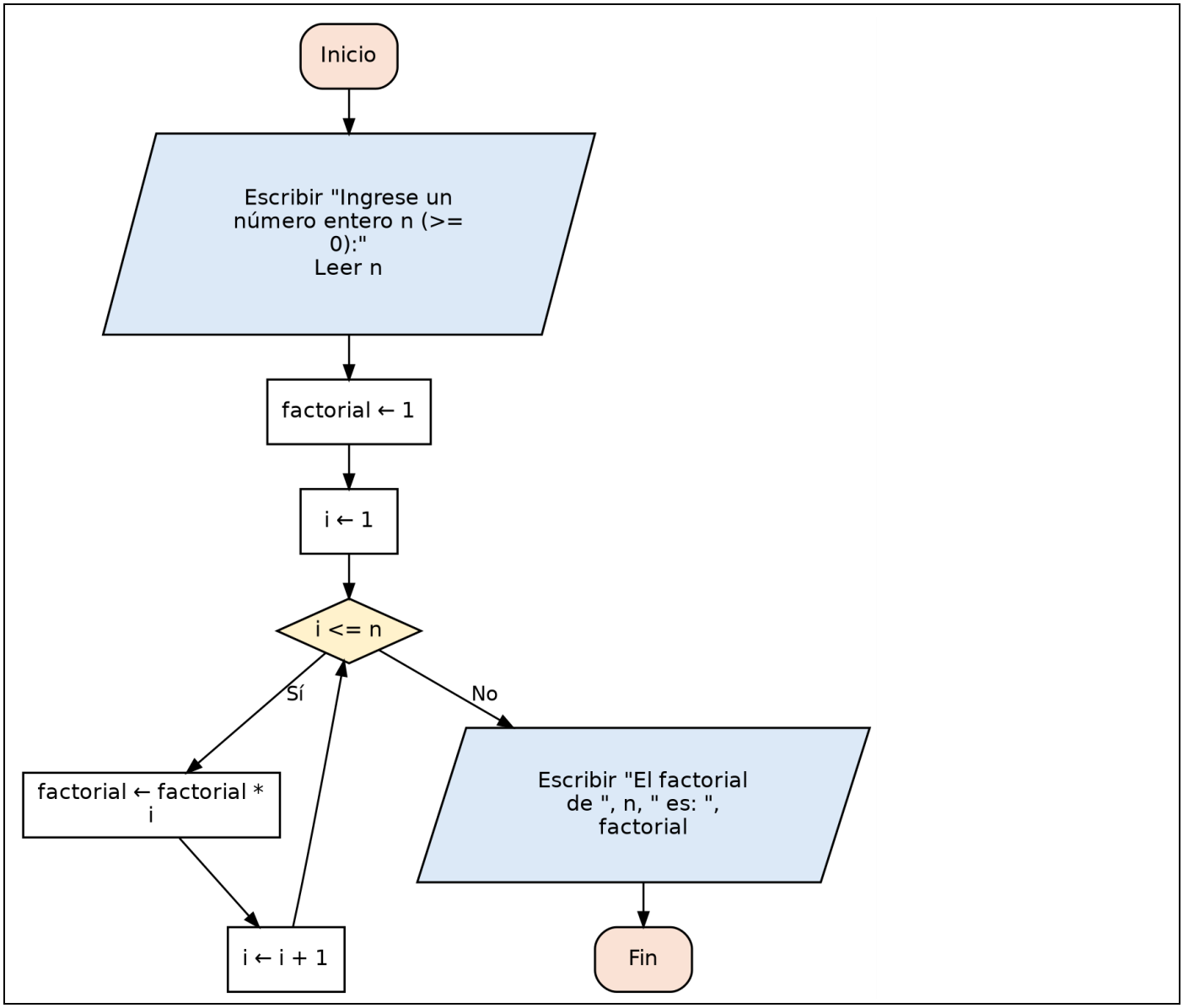
Seudocódigo

```
Proceso CalcularFactorial
   Definir n, i Como Entero
   Definir factorial Como Entero
   Escribir "Ingrese un número entero n (>= 0):"
   Leer n
   factorial <- 1
   Para i <- 1 Hasta n Hacer
      factorial <- factorial * i
   FinPara
   Escribir "El factorial de ", n, " es: ", factorial
FinProceso
```

Prueba de escritorio

i	factorial (antes)	Operación	factorial (después)
1	1	1 * 1	1
2	1	1 * 2	2
3	2	2 * 3	6
4	6	6 * 4	24

Diagrama de flujo



Código fuente

```
#include <stdio.h>

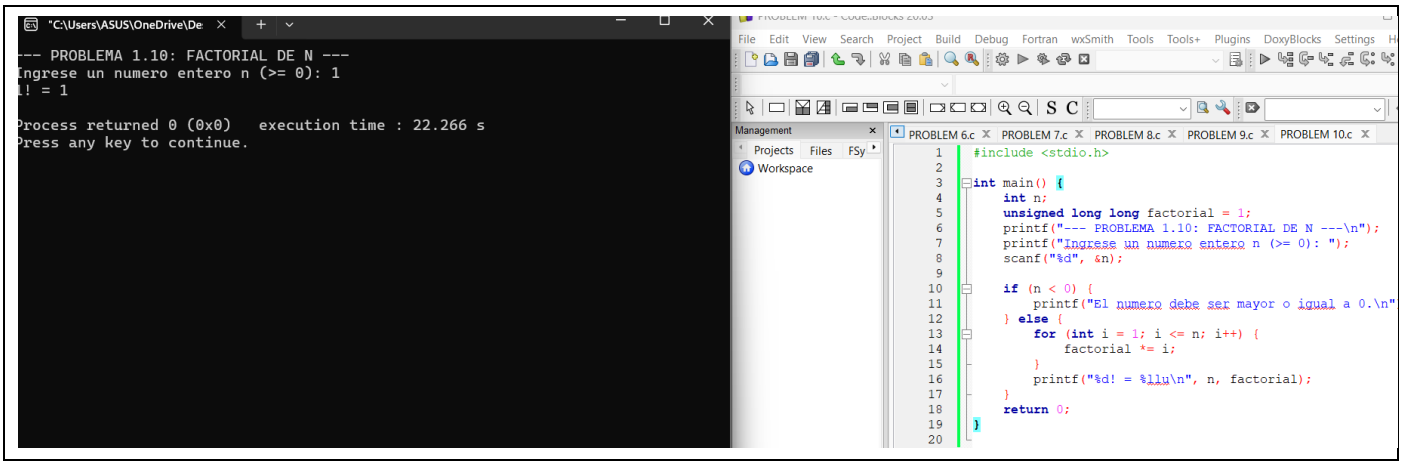
int main()
{
    int n, i;
    long long factorial;

    printf("Ingrese un numero entero n (>= 0): ");
    scanf("%d", &n);

    factorial = 1;
    for (i = 1; i <= n; i++)
    {
        factorial = factorial * i;
    }
```

```
    printf("El factorial de %d es: %lld\n", n, factorial);
    return 0;
}
```

Pantallazo de Code::Blocks



11- Serie de Fibonacci

Desarrolla un programa que lea un entero n ≥ 1 y muestre los primeros n términos de la serie de Fibonacci (0, 1, 1, 2, 3, 5, 8, ...), donde cada término es la suma de los dos anteriores.

Ejemplo: n = 7 → 0 1 1 2 3 5 8.

Pista: Necesitas tres variables: el término anterior, el actual y el siguiente.

Para pensar: ¿Cómo cambia tu algoritmo si solo debes mostrar el término número n y no toda la serie?

Análisis del problema

- **Entrada:** Un número entero n ≥ 1 que indica la cantidad de términos a generar.
- **Proceso:** Utilizar tres variables (anterior, actual y siguiente) dentro de un ciclo de n iteraciones para calcular la suma de los dos términos previos.
- **Salida:** Los primeros n términos de la serie de Fibonacci.
- **Restricciones:** n debe ser un entero mayor o igual a 1.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	Ingresado por teclado	Entero
Variable auxiliar	anterior	0	Entero
Variable auxiliar	actual	1	Entero
Variable auxiliar	siguiente	calculado	Entero
Variable auxiliar	i	1	Entero
Dato de salida	termino_fibonacci	Ingreso en pantalla	Entero

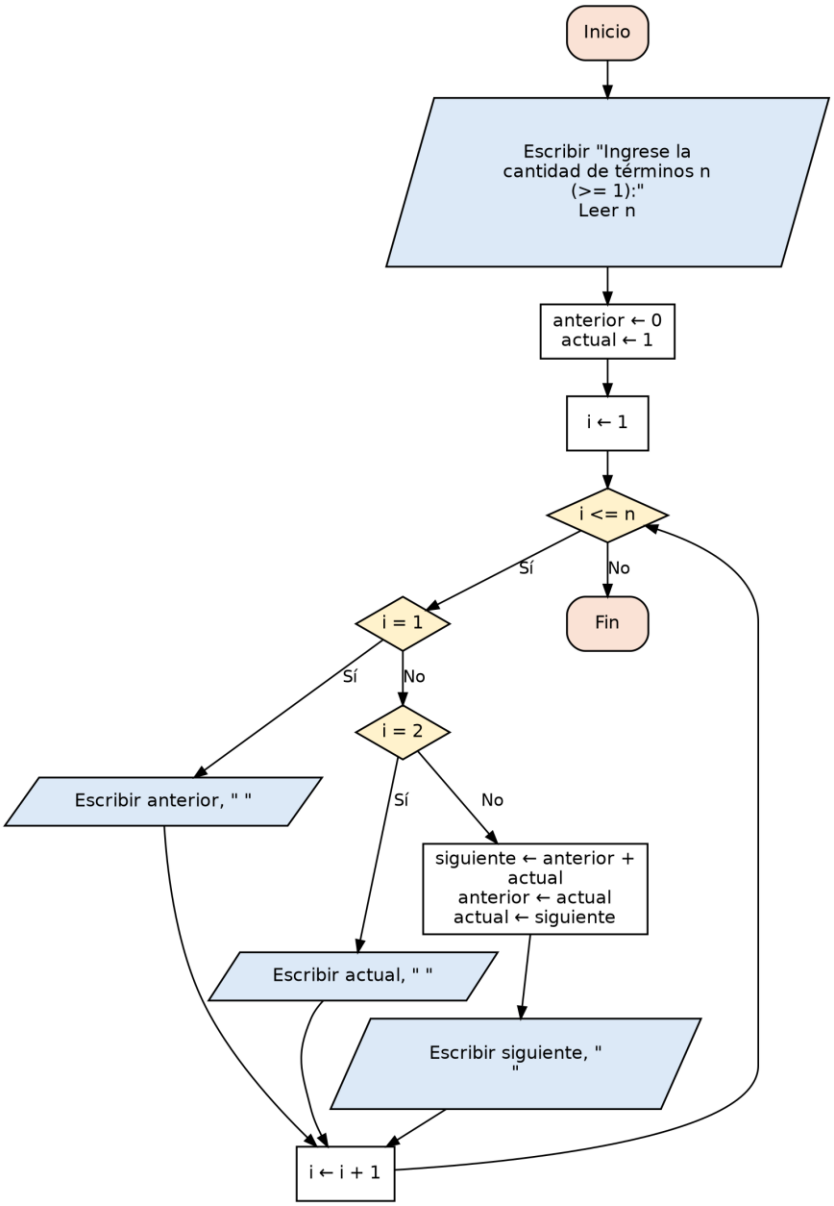
Seudocódigo

```
Proceso SerieFibonacci
  Definir n, anterior, actual, siguiente, i Como Entero
  Escribir "Ingrese la cantidad de términos n (>= 1):"
  Leer n
  anterior <- 0
  actual <- 1
  Escribir "Serie de Fibonacci:"
  Para i <- 1 Hasta n Hacer
    Si i = 1 Entonces
      Escribir Sin Saltar anterior, " "
    Sino
      Si i = 2 Entonces
        Escribir Sin Saltar actual, " "
      Sino
        siguiente <- anterior + actual
        anterior <- actual
        actual <- siguiente
        Escribir Sin Saltar siguiente, " "
      FinSi
    FinSi
  FinPara
  Escribir ""
FinProceso
```

Prueba de escritorio

i	anterior	actual	siguiente	Salida en pantalla
1	0	1	-	0
2	0	1	-	1
3	1	1	0 + 1 = 1	1
4	1	2	1 + 1 = 2	2
5	2	3	1 + 2 = 3	3

DIAGRAMA DE FLUJO



Código fuente

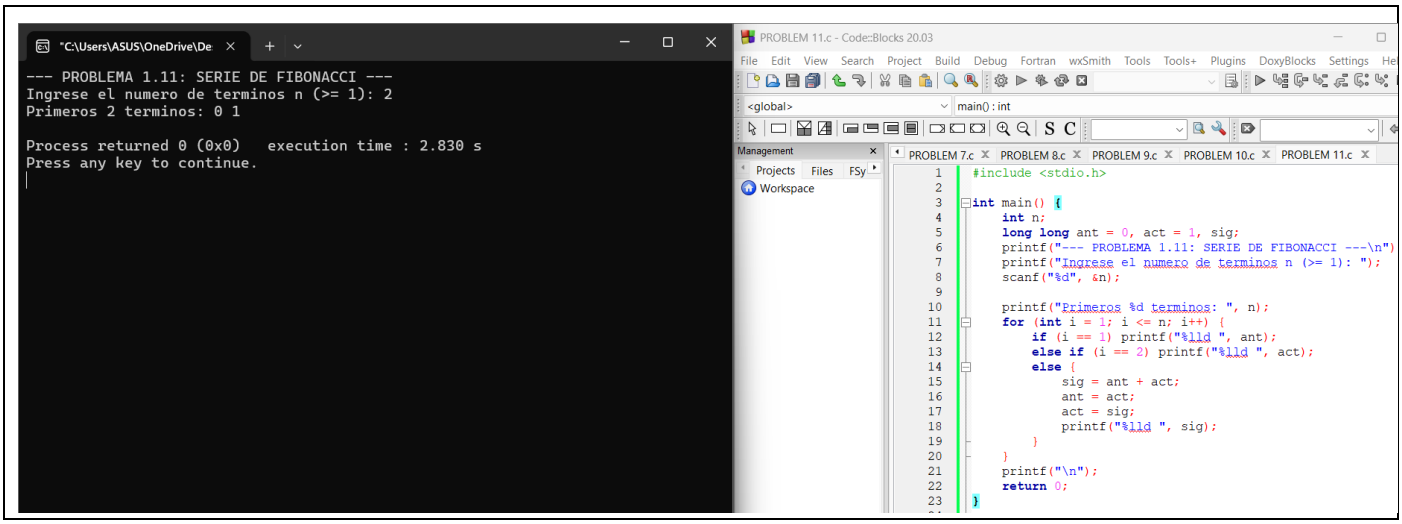
```c
#include <stdio.h>

int main()
{
    int n, anterior, actual, siguiente, i;

    printf("Ingrese la cantidad de terminos n (>= 1): ");
    scanf("%d", &n);

    anterior = 0;
    actual = 1;
    for (i = 1; i <= n; i++)
    {
        if (i == 1)
        {
            printf("%d ", anterior);
        }
        else if (i == 2)
        {
            printf("%d ", actual);
        }
        else
        {
            siguiente = anterior + actual;
            anterior = actual;
            actual = siguiente;
            printf("%d ", siguiente);
        }
    }
    printf("\n");
    return 0;
}
```

Pantallazo de Code::Blocks



12- Suma de dígitos

Desarrolla un programa que lea un entero positivo y calcule la suma de sus dígitos.
Ejemplo: 4 825 → 4 + 8 + 2 + 5 = 19.
Pista: El resto de dividir entre 10 (n % 10) te da el último dígito; la división entera entre 10 lo elimina.
Para pensar: ¿Cuántas veces se repite tu ciclo para un número de k dígitos? Justifica tu respuesta.

Análisis del problema

- **Entrada:** Un número entero positivo.
- **Proceso:** Extraer el último dígito mediante módulo 10 (% 10), sumarlo a un acumulador y eliminarlo mediante división entera entre 10 (/ 10) de forma repetitiva hasta reducir el número a 0.
- **Salida:** La suma total de los dígitos del número.
- **Restricciones:** El número de entrada debe ser un entero positivo.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	numero	Ingreso por teclado	Entero
Variable auxiliar	digito	calculado	Entero
Dato de salida	suma	0	Entero

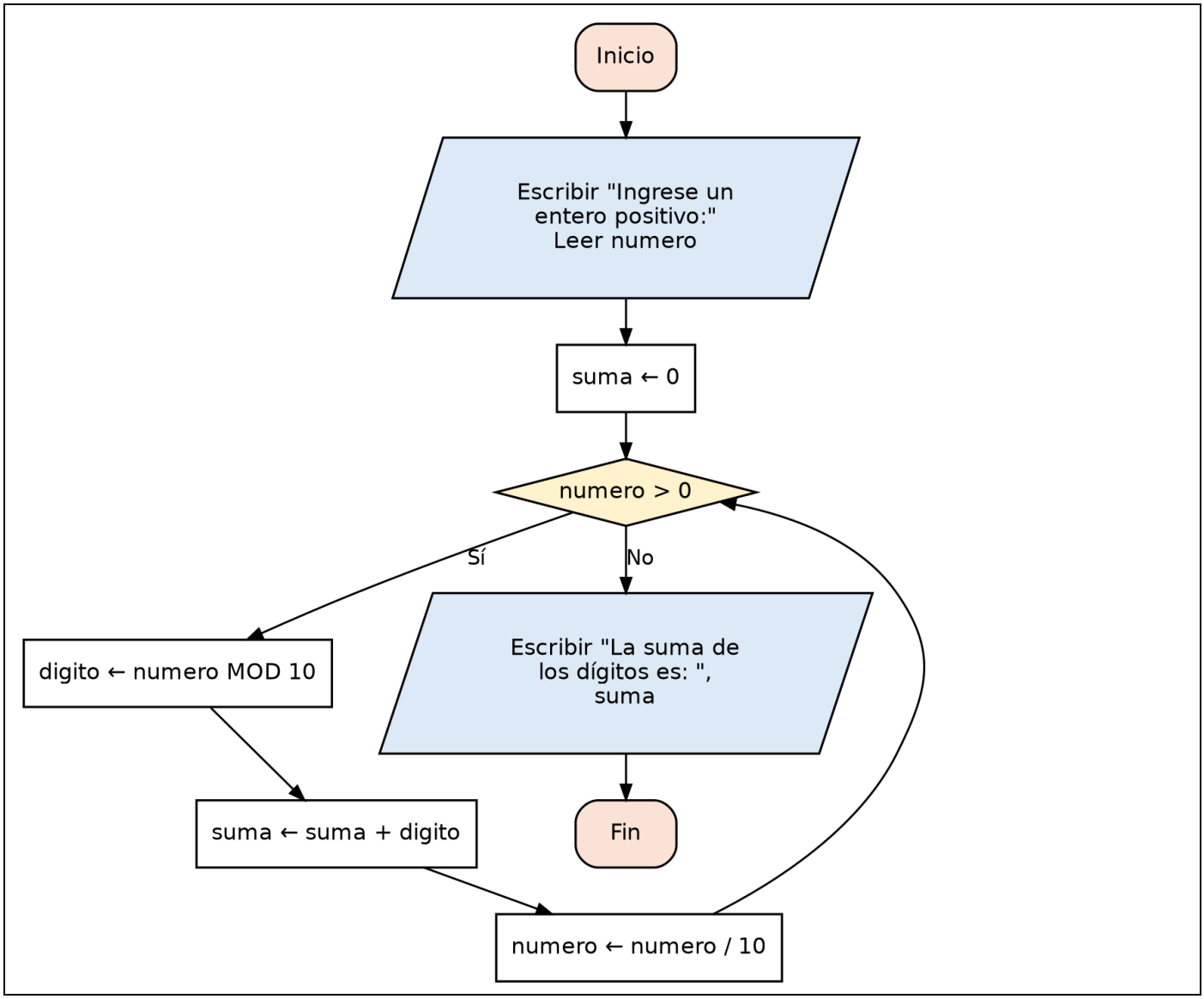
Seudocódigo

```
Proceso SumaDigitos
   Definir numero, suma, digito Como Entero
   Escribir "Ingrese un entero positivo:"
   Leer numero
   suma <- 0
   Mientras numero > 0 Hacer
      digito <- numero MOD 10
      suma <- suma + digito
      numero <- numero / 10
   FinMientras
   Escribir "La suma de los dígitos es: ", suma
FinProceso
```

Prueba de escritorio

Iteración	numero (antes)	digito (num MOD 10)	suma	numero (después)
1	352	2	0 + 2 = 2	35
2	35	5	2 + 5 = 7	3
3	3	3	7 + 3 = 10	0 (Fin)

Diagrama de flujo



Código fuente

```
#include <stdio.h>

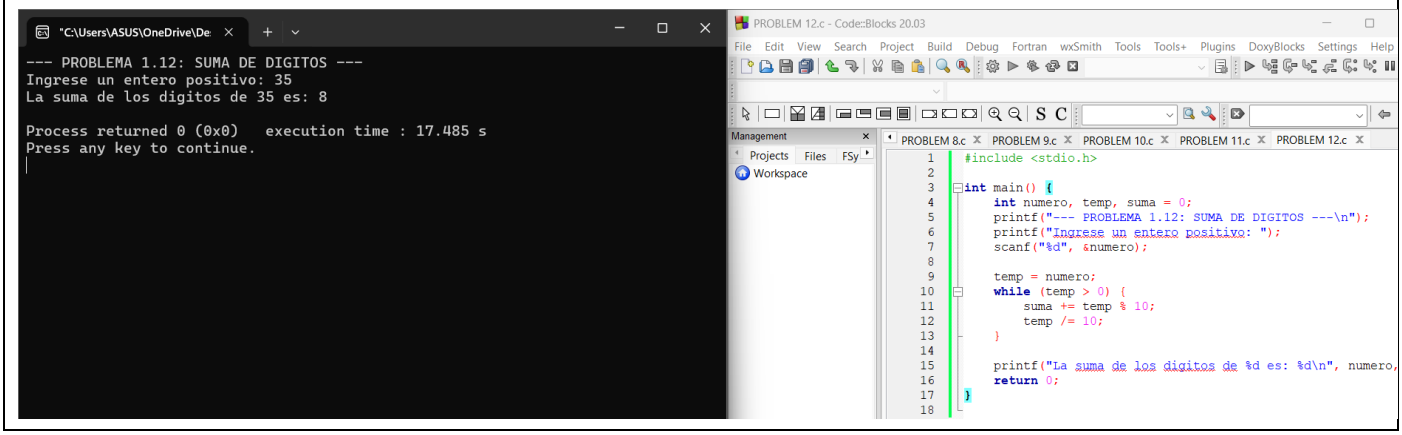
int main()
{
    int numero, suma, digito;

    printf("Ingrese un entero positivo: ");
    scanf("%d", &numero);

    suma = 0;
    while (numero > 0)
    {
        digito = numero % 10;
        suma = suma + digito;
        numero = numero / 10;
    }
```

```
    printf("La suma de los digitos es: %d\n", suma);
    return 0;
}
```

Pantallazo de Code::Blocks



13- Número capicúa

Desarrolla un programa que lea un entero positivo y determine si es capicúa, es decir, si se lee igual de izquierda a derecha que de derecha a izquierda.

Ejemplos: 12321 es capicúa; 1231 no lo es.

Pista: Construye el número invertido con el mismo recurso del ejercicio 1.12 y compáralo con el original.

Para pensar: ¿Qué casos de prueba elegirías para estar seguro/a de que tu solución es correcta (por ejemplo, números de un solo dígito o que terminan en 0)?

Análisis del problema

- **Entrada:** Un número entero positivo.
- **Proceso:** Invertir el número dígito por dígito utilizando las operaciones de módulo y división entera entre 10, y comparar el valor invertido con el número original.
- **Salida:** Un mensaje indicando si el número es capicúa o no.
- **Restricciones:** El número debe ser entero positivo.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	numero	Ingresado por teclado	Entero
Variable auxiliar	auxiliar	Copia de numero	Entero
Variable auxiliar	invertido	0	entero
Variable auxiliar	digito	calculado	Entero
Dato de salida	resultado	evaluado	Lógico

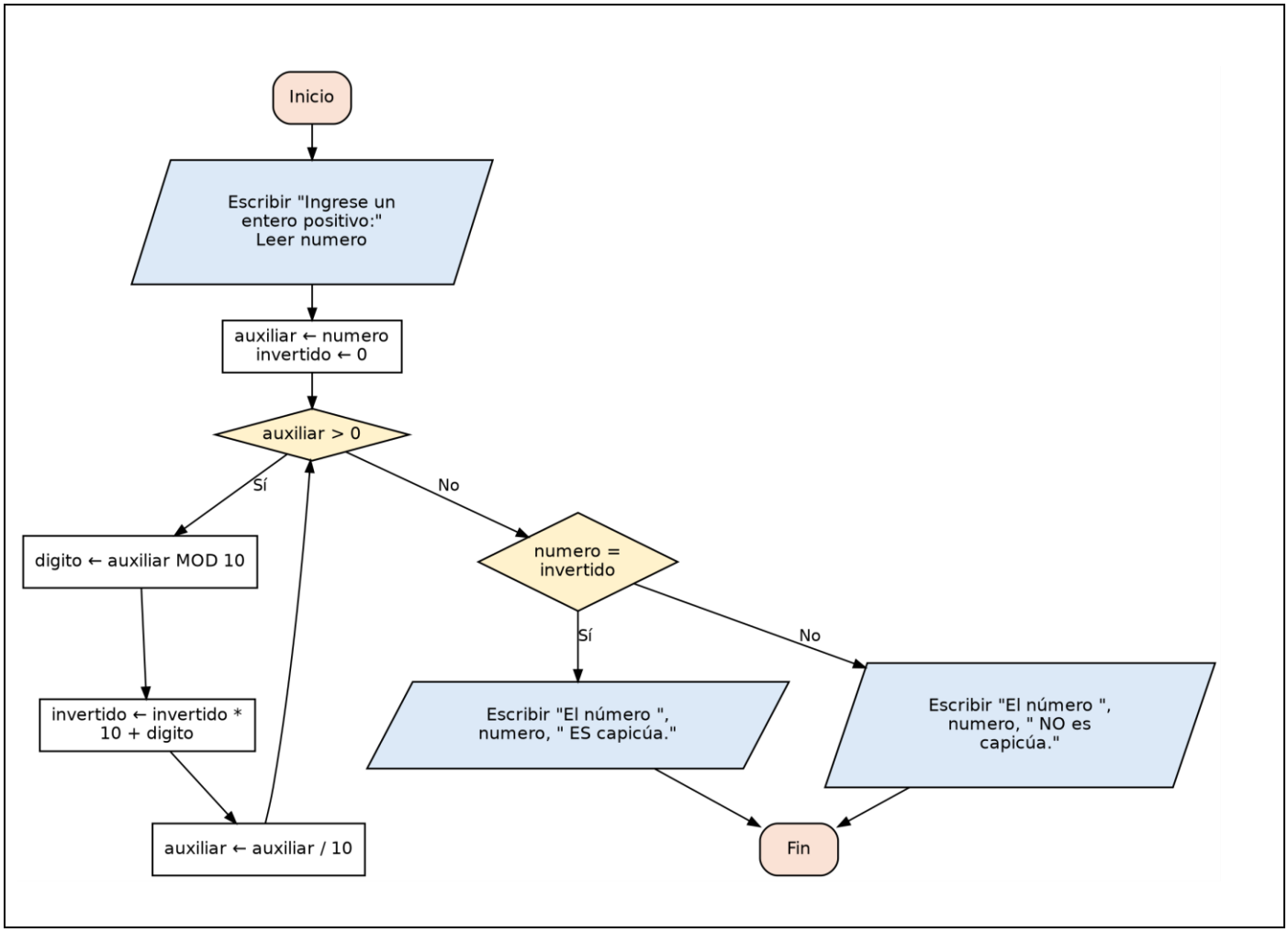
Seudocódigo

```
Proceso NumeroCapicua
   Definir numero, auxiliar, invertido, digito Como Entero
   Escribir "Ingrese un entero positivo:"
   Leer numero
   auxiliar <- numero
   invertido <- 0
   Mientras auxiliar > 0 Hacer
      digito <- auxiliar MOD 10
      invertido <- invertido * 10 + digito
      auxiliar <- auxiliar / 10
   FinMientras
   Si numero = invertido Entonces
      Escribir "El número ", numero, " ES capicúa."
   Sino
      Escribir "El número ", numero, " NO es capicúa."
   FinSi
FinProceso
```

Prueba de escritorio

auxiliar (antes)	digito	invertido (después)	auxiliar (después)
121	1	0 * 10 + 1 = 1	12
12	2	1 * 10 + 2 = 12	1
1	1	12 * 10 + 1 = 121	0 (121 = 121 -> ES capicúa)

Diagrama de flujo



Código fuente

```
#include <stdio.h>

int main()
{
    int numero, auxiliar, invertido, digito;

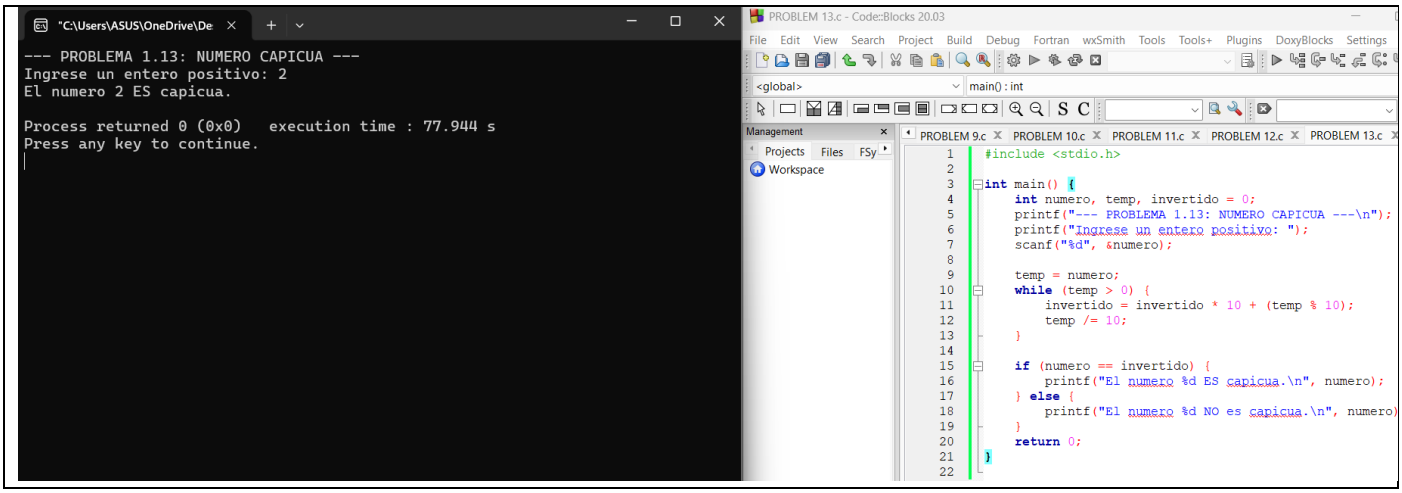
    printf("Ingrese un entero positivo: ");
    scanf("%d", &numero);

    auxiliar = numero;
    invertido = 0;
    while (auxiliar > 0)
    {
        digito = auxiliar % 10;
        invertido = invertido * 10 + digito;
        auxiliar = auxiliar / 10;
    }

    if (numero == invertido)
    {
        printf("El numero %d ES capicua.\n", numero);
    }
```

```
    else
    {
        printf("El numero %d NO es capicua.\n", numero);
    }
    return 0;
}
```

Pantallazo de Code::Blocks



14- Máximo común divisor (algoritmo de Euclides)

Desarrolla un programa que lea dos enteros positivos a y b y calcule su máximo común divisor con el algoritmo de Euclides: mientras b ≠ 0, reemplaza (a, b) por (b, a % b). Al terminar, a es el MCD.

Ejemplo: MCD(48, 18) = 6.

Pista: Haz la prueba de escritorio con (48, 18): los pares sucesivos son (48, 18), (18, 12), (12, 6) y (6, 0).

Para pensar: Compara este método con probar todos los divisores desde 1 hasta el menor de los dos números. ¿Cuál hace menos operaciones y por qué?

Análisis del problema

- **Entrada:** Dos números enteros positivos a y b.
- **Proceso:** Aplicar el algoritmo de Euclides mediante un ciclo: mientras b/0, actualizar los valores asignando a *a* el valor de b y a b el residuo de a % b.
- **Salida:** El Máximo Común Divisor (MCD) de ambos números.
- **Restricciones:** Los números de entrada deben ser enteros positivos.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	a	Ingresado por teclado	Entero
Dato de entrada	b	Ingresado por teclado	Entero
Variable auxiliar	temporal	Sin inicializar	Entero
Dato de salida	mcd	Valor final de a	Entero

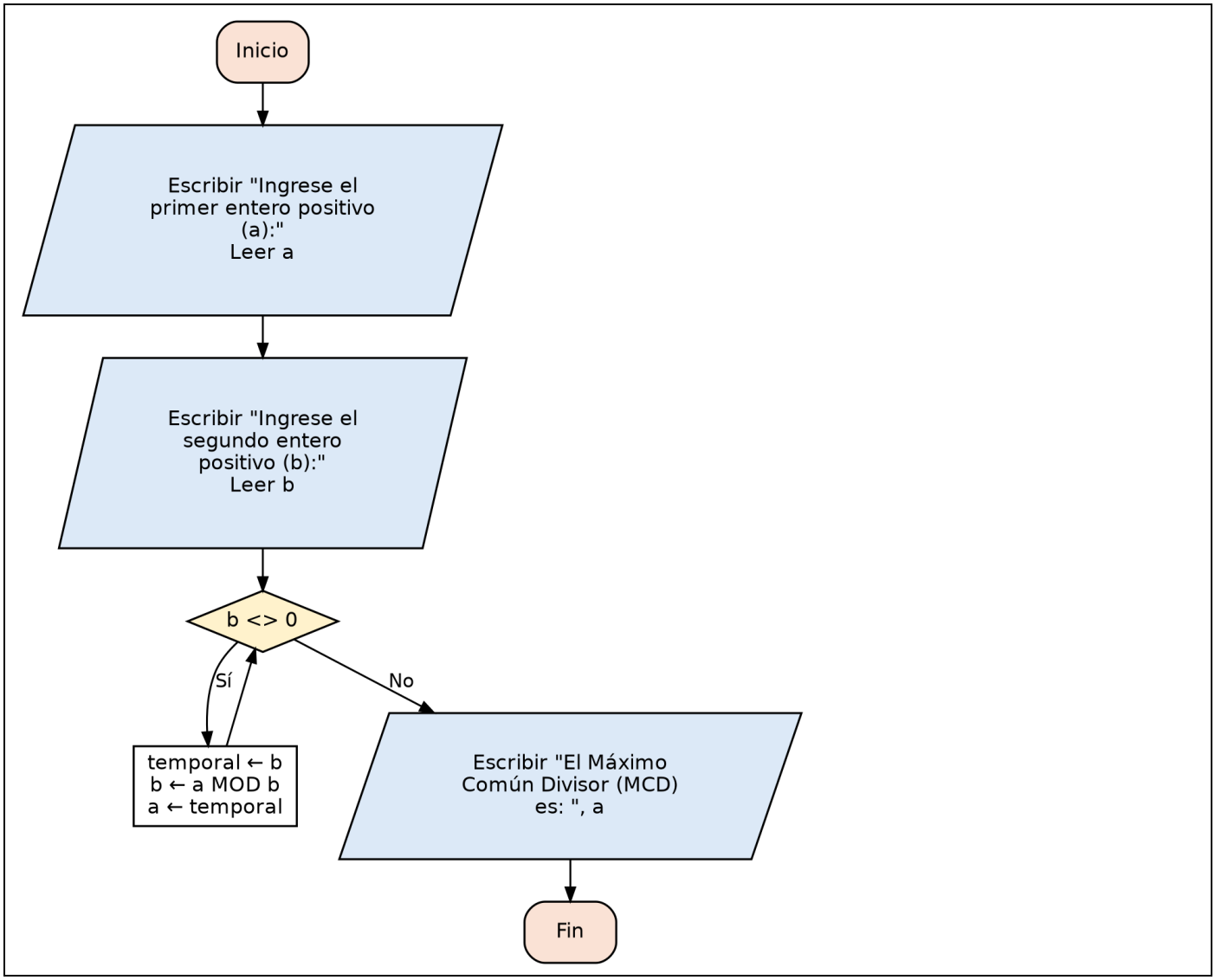
Seudocódigo

```
Proceso MaximoComunDivisor
   Definir a, b, temporal Como Entero
   Escribir "Ingrese el primer entero positivo (a):"
   Leer a
   Escribir "Ingrese el segundo entero positivo (b):"
   Leer b
   Mientras b <> 0 Hacer
      temporal <- b
      b <- a MOD b
      a <- temporal
   FinMientras
   Escribir "El Máximo Común Divisor (MCD) es: ", a
FinProceso
```

Prueba de escritorio

Iteración	a (antes)	b (antes)	b <> 0?	b (nuevo: a MOD b)	a (nuevo: temporal)
1	48	18	Sí	48 MOD 18 = 12	18
2	18	12	Sí	18 MOD 12 = 6	12
3	12	6	Sí	12 MOD 6 = 0	6
4	6	0	No (Fin)	-	MCD = 6

Diagrama de flujo



Código fuente

```
#include <stdio.h>

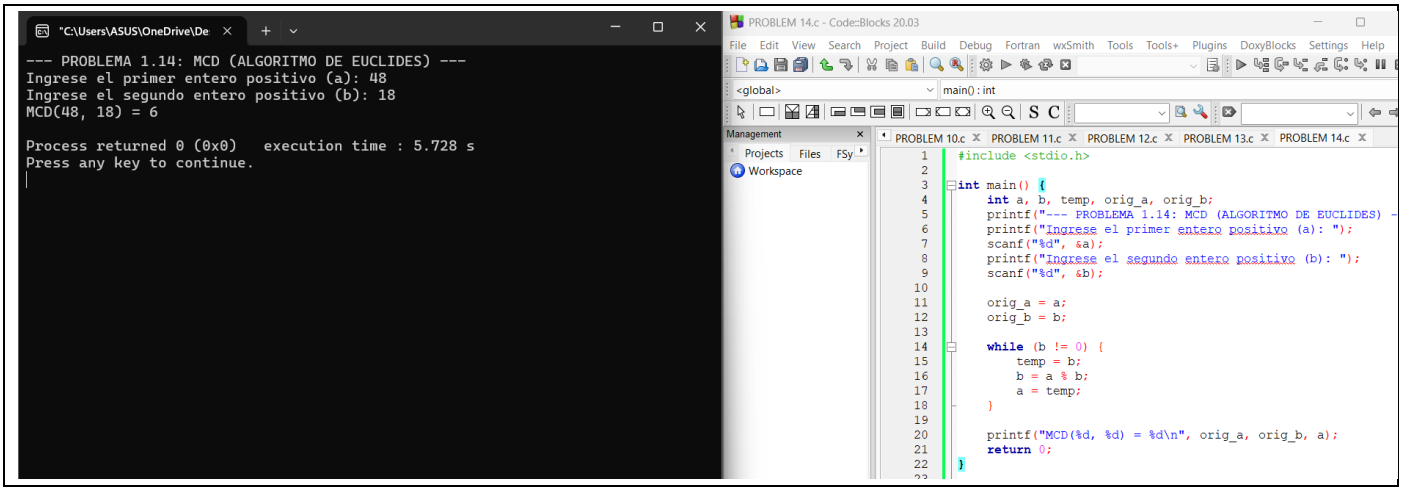
int main()
{
    int a, b, temporal;

    printf("Ingrese el primer entero positivo (a): ");
    scanf("%d", &a);
    printf("Ingrese el segundo entero positivo (b): ");
    scanf("%d", &b);

    while (b != 0)
    {
        temporal = b;
        b = a % b;
        a = temporal;
    }
```

```
    printf("El Maximo Comun Divisor (MCD) es: %d\n", a);
    return 0;
}
```

Pantallazo de Code::Blocks



15- Año bisiesto

Desarrolla un programa que lea un año y determine si es bisiesto. Un año es bisiesto si es divisible entre 4, excepto los años divisibles entre 100, que solo lo son si también son divisibles entre 400.

Ejemplos: 2024 sí; 1900 no; 2000 sí.

Pista: Combina los operadores &&, || y % en una sola condición, o utiliza condiciones anidadas.

Para pensar: ¿Es equivalente escribir las condiciones en otro orden? Prueba con 1900 y 2000 para comprobarlo.

Análisis del problema

- **Entrada:** Un número entero que representa un año.
- **Proceso:** Evaluar la condición lógica: un año es bisiesto si es divisible entre 4 (año % 4 == 0) y no es divisible entre 100 (año % 100 != 0), o bien si es divisible entre 400 (año % 400 == 0).
- **Salida:** Un mensaje indicando si el año es bisiesto o no.
- **Restricciones:** El año debe ser un valor entero positivo válido.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	anio	Ingresado por teclado	Entero
Dato de salida	es_bisiesto	Evaluado	Lógico

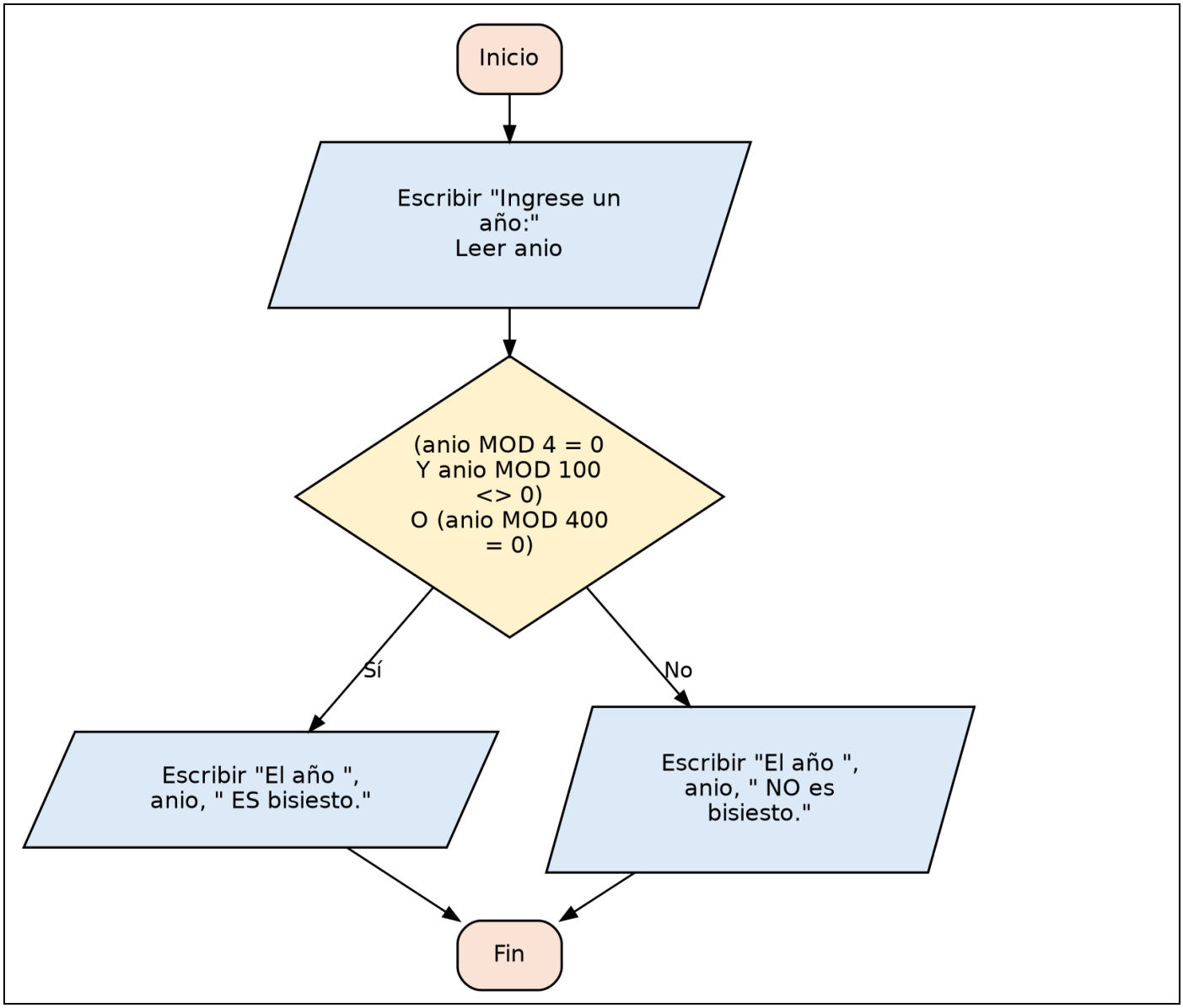
Seudocódigo

```
Proceso AnioBisiesto
  Definir anio Como Entero
  Escribir "Ingrese un año:"
  Leer anio
  Si (anio MOD 4 = 0 Y anio MOD 100 <> 0) O (anio MOD 400 = 0) Entonces
    Escribir "El año ", anio, " ES bisiesto."
  Sino
    Escribir "El año ", anio, " NO es bisiesto."
  FinSi
FinProceso
```

Prueba de escritorio

Prueba (anio)	Condición (anio % 4 == 0 Y anio % 100 != 0)	Condición (anio % 400 == 0)	Resultado final
2024	Verdadero Y Verdadero -> Verdadero	Falso	ES bisiesto
1900	Verdadero Y Falso -> Falso	Falso	NO es bisiesto
2000	Verdadero Y Falso -> Falso	Verdadero	ES bisiesto

Diagrama de flujo



Código fuente

```
#include <stdio.h>

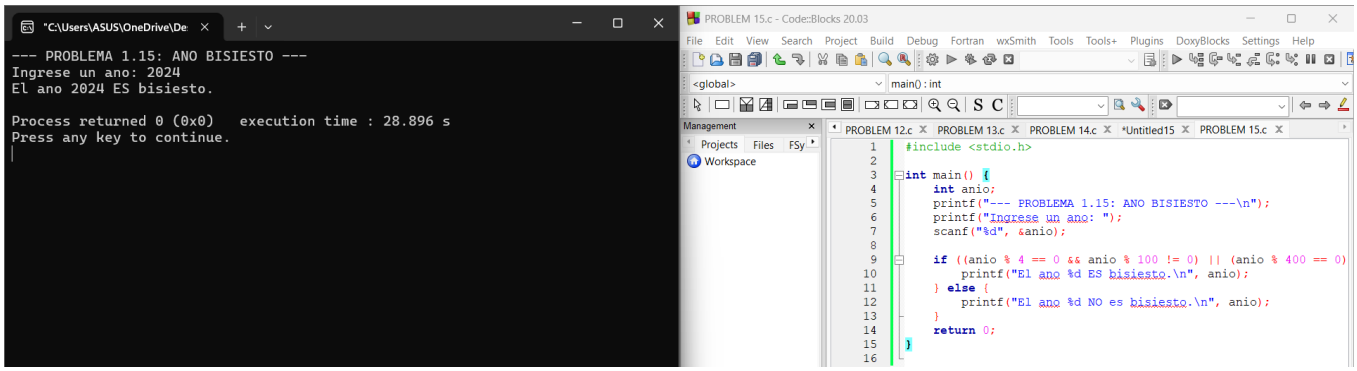
int main()
{
    int anio;

    printf("Ingrese un anio: ");
    scanf("%d", &anio);

    if ((anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0))
    {
        printf("El anio %d ES bisiesto.\n", anio);
    }
    else
    {
        printf("El anio %d NO es bisiesto.\n", anio);
```

```
    }
    return 0;
}
```

Pantallazo de Code::Blocks



16-Número perfecto

Desarrolla un programa que lea un entero positivo y determine si es perfecto, es decir, si es igual a la suma de sus divisores propios (los menores que él).

Ejemplo: 28 = 1 + 2 + 4 + 7 + 14, por lo tanto es perfecto.

Pista: Recorre los posibles divisores desde 1 hasta n – 1 y acumula los que cumplan n % d == 0.

Para pensar: ¿Hasta qué valor es realmente necesario recorrer los divisores? Explica cómo reducirías el número de vueltas.

Análisis del problema

- **Entrada:** Un número entero positivo.
- **Proceso:** Recorrer los posibles divisores desde 1 hasta n-1, identificar aquellos que dividen exactamente a n (n % d == 0), sumarlos y verificar si la suma equivale al número original.
- **Salida:** Un mensaje indicando si el número es perfecto o no.
- **Restricciones:** El número de entrada debe ser un entero positivo.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	Ingresado por teclado	Entero
Variable auxiliar	d	1	Entero
Variable auxiliar	suma_divisores	0	Entero
Dato de salida	es_perfecto	evaluado	Lógico

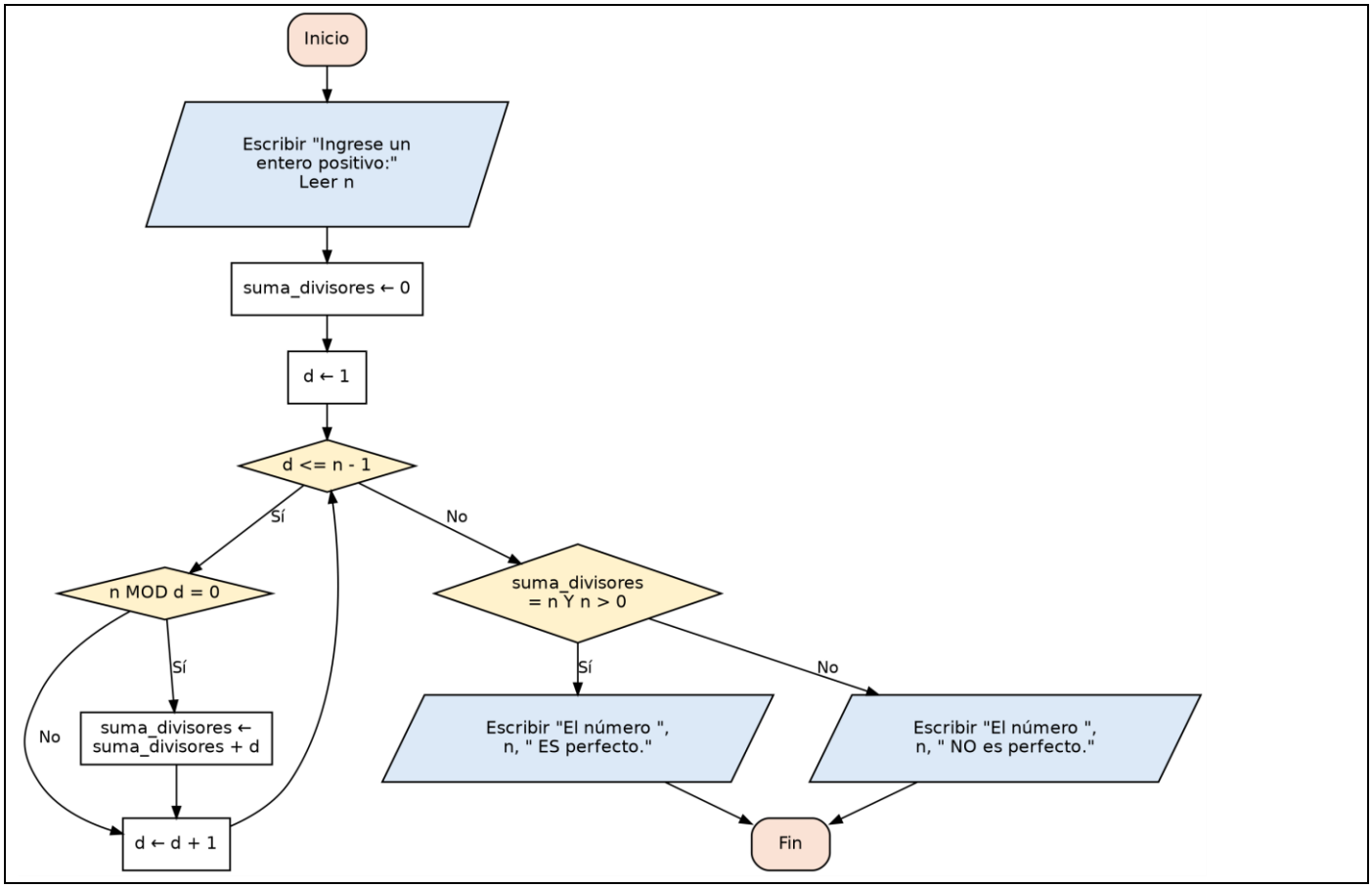
Seudocódigo

```
Proceso NumeroPerfecto
  Definir n, d, suma_divisores Como Entero
  Escribir "Ingrese un entero positivo:"
  Leer n
  suma_divisores <- 0
  Para d <- 1 Hasta n - 1 Hacer
    Si n MOD d = 0 Entonces
      suma_divisores <- suma_divisores + d
    FinSi
  FinPara
  Si suma_divisores = n Y n > 0 Entonces
    Escribir "El número ", n, " ES perfecto."
  Sino
    Escribir "El número ", n, " NO es perfecto."
  FinSi
FinProceso
```

Prueba de escritorio

d	n MOD d = 0?	suma divisores (después)
1	6 MOD 1 = 0 (Sí)	0 + 1 = 1
2	6 MOD 2 = 0 (Sí)	1 + 2 = 3
3	6 MOD 3 = 0 (Sí)	3 + 3 = 6
4	6 MOD 4 = 2 (No)	6
5	6 MOD 5 = 1 (No)	6 (Suma 6 = 6 -> ES perfecto)

Diagrama de flujo



Código fuente

```
#include <stdio.h>

int main()
{
    int n, d, suma_divisores;

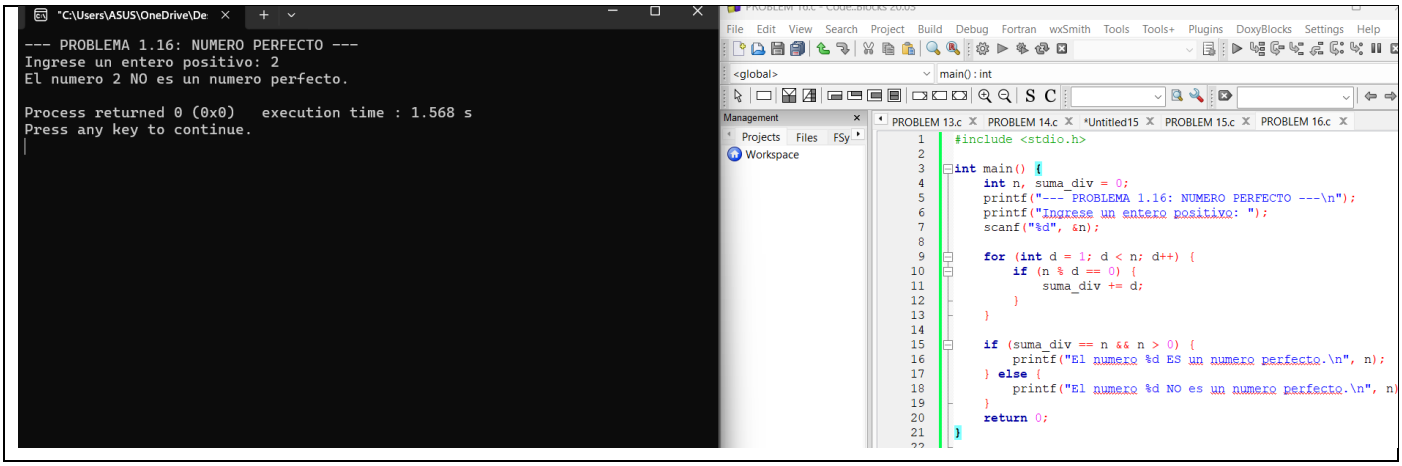
    printf("Ingrese un entero positivo: ");
    scanf("%d", &n);

    suma_divisores = 0;
    for (d = 1; d <= n - 1; d++)
    {
        if (n % d == 0)
        {
            suma_divisores = suma_divisores + d;
        }
    }

    if (suma_divisores == n && n > 0)
    {
        printf("El numero %d ES perfecto.\n", n);
    }
    else
    {
        printf("El numero %d NO es perfecto.\n", n);
    }
```

```
    return 0;
}
```

Pantallazo de Code::Blocks



17- Conversión de decimal a binario

Desarrolla un programa que lea un entero positivo en base 10 y muestre su representación en binario.

Ejemplo: 13 → 1101.

Pista: Divide entre 2 sucesivamente y guarda los restos; el resultado se lee de abajo hacia arriba.

Para pensar: Los restos se obtienen en orden inverso al que se leen. ¿Cómo resolverías ese detalle sin usar arreglos?

Análisis del problema

- **Entrada:** Un número entero positivo en base 10.
- **Proceso:** Dividir sucesivamente el número entre 2, almacenando o imprimiendo los restos de cada división hasta que el cociente sea 0. El resultado binario se lee en orden inverso.
- **Salida:** La representación del número en sistema binario.
- **Restricciones:** El número debe ser entero positivo mayor o igual a 0.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	decimal	Ingresado por teclado	Entero
Variable auxiliar	resto	Calculado	Entero
Variable auxiliar	multiplicador	1	Entero
Dato de salida	binario	0	Entero

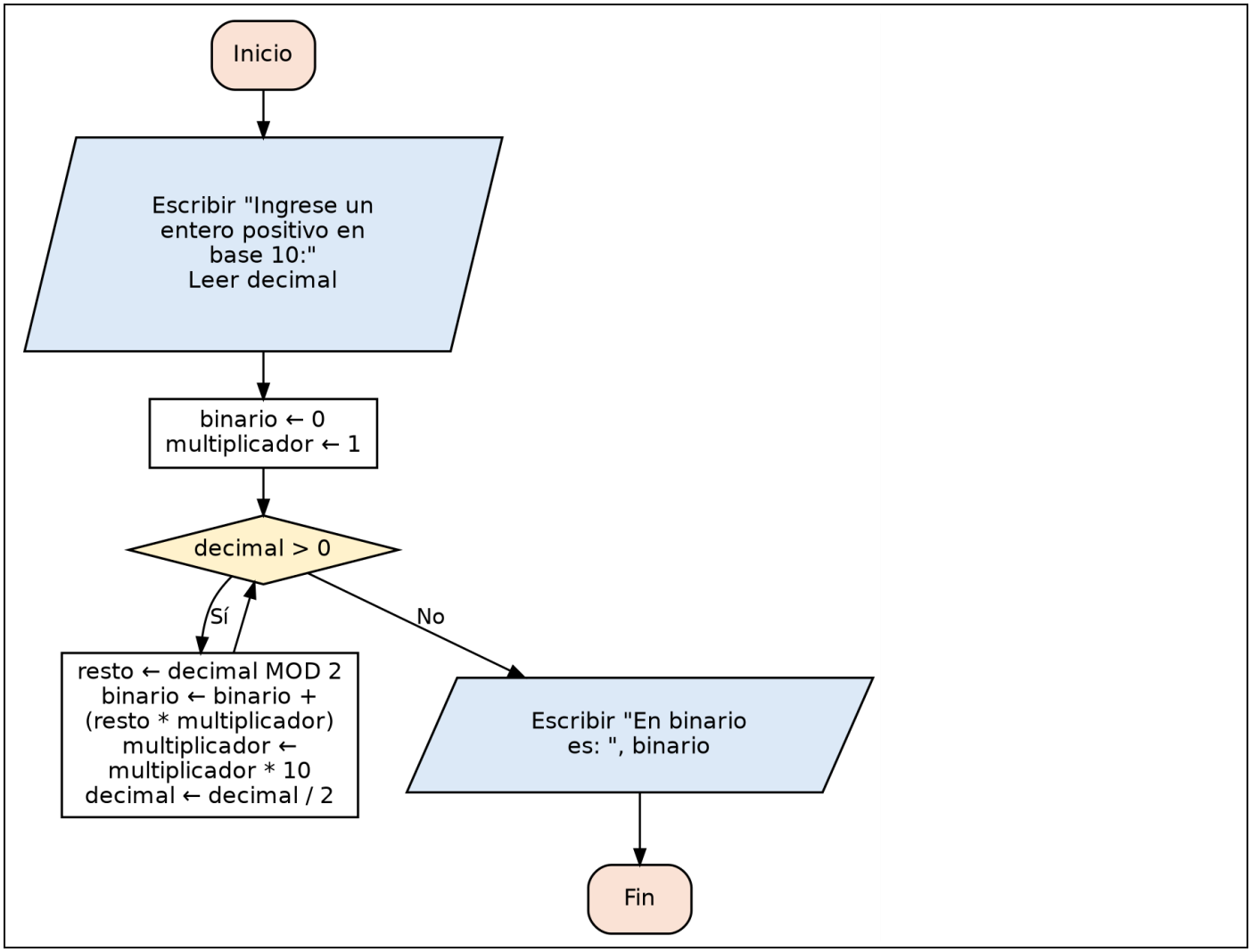
Seudocódigo

```
Proceso DecimalABinario
   Definir decimal, resto, binario, multiplicador Como Entero
   Escribir "Ingrese un entero positivo en base 10:"
   Leer decimal
   binario <- 0
   multiplicador <- 1
   Si decimal = 0 Entonces
      binario <- 0
   Sino
      Mientras decimal > 0 Hacer
         resto <- decimal MOD 2
         binario <- binario + (resto * multiplicador)
         multiplicador <- multiplicador * 10
         decimal <- decimal / 2
      FinMientras
   FinSi
   Escribir "La representación en binario es: ", binario
FinProceso
```

Prueba de escritorio

decimal (antes)	resto (dec MOD 2)	multiplicador	binario (después)	decimal (después)
5	1	1	0 + (1 * 1) = 1	2
2	0	10	1 + (0 * 10) = 1	1
1	1	100	1 + (1 * 100) = 101	0 (Binario final: 101)

Diagrama de flujo



Código fuente

```
#include <stdio.h>

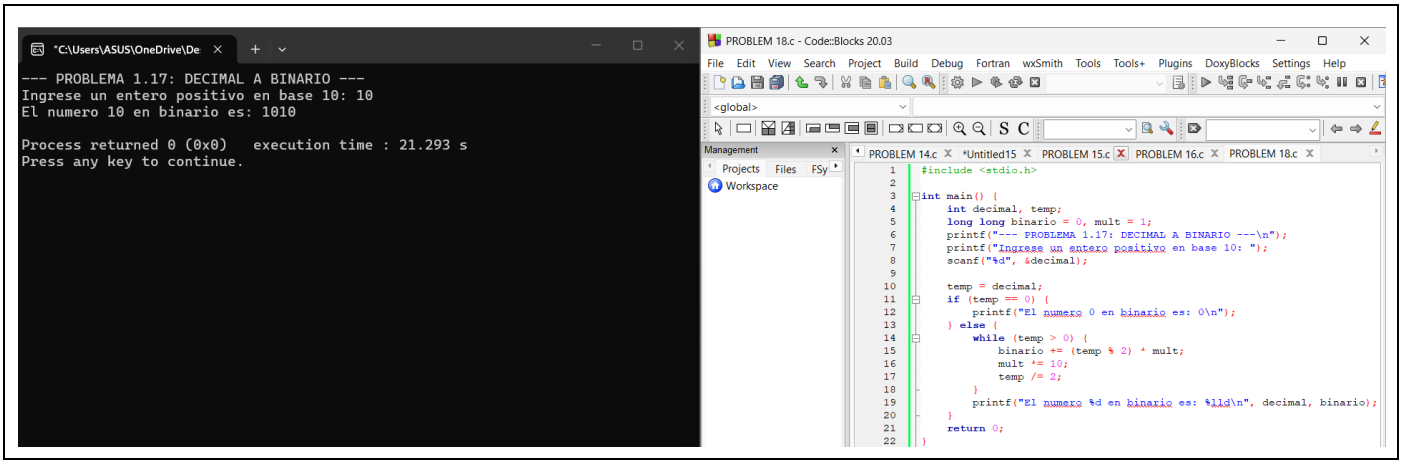
int main()
{
    int decimal, resto;
    long long binario, multiplicador;

    printf("Ingrese un entero positivo en base 10: ");
    scanf("%d", &decimal);

    binario = 0;
    multiplicador = 1;
    while (decimal > 0)
    {
        resto = decimal % 2;
        binario = binario + (resto * multiplicador);
        multiplicador = multiplicador * 10;
        decimal = decimal / 2;
    }
```

```
    printf("En binario es: %lld\n", binario);
    return 0;
}
```

Pantallazo de Code::Blocks



18-Triángulo de asteriscos

Desarrolla un programa que lea un entero n ≥ 1 y dibuje por pantalla un triángulo rectángulo de n filas, donde la fila i tiene i asteriscos.

Ejemplo con n = 4:

```
*
**
***
****
```

Pista: Usa un ciclo dentro de otro: el externo controla las filas y el interno, los asteriscos de cada fila.

Para pensar: Si ahora quisieras un triángulo centrado (una pirámide), ¿qué parte de tu algoritmo cambiaría y qué parte se mantendría?

Análisis del problema

- **Entrada:** Un número entero n ≥ 1 que representa el número de filas del triángulo.
- **Proceso:** Utilizar dos ciclos anidados: un ciclo externo para controlar las filas de 1 hasta n, y un ciclo interno para imprimir la cantidad de asteriscos correspondiente al número de la fila actual.
- **Salida:** Un triángulo rectángulo formado por asteriscos en pantalla.
- **Restricciones:** n debe ser un entero mayor o igual a 1.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	Ingresado por teclado	Entero
Variable auxiliar	i	1	Entero
Variable auxiliar	j	1	Entero
Dato de salida	asterisco	Impreso en pantalla	Carácter

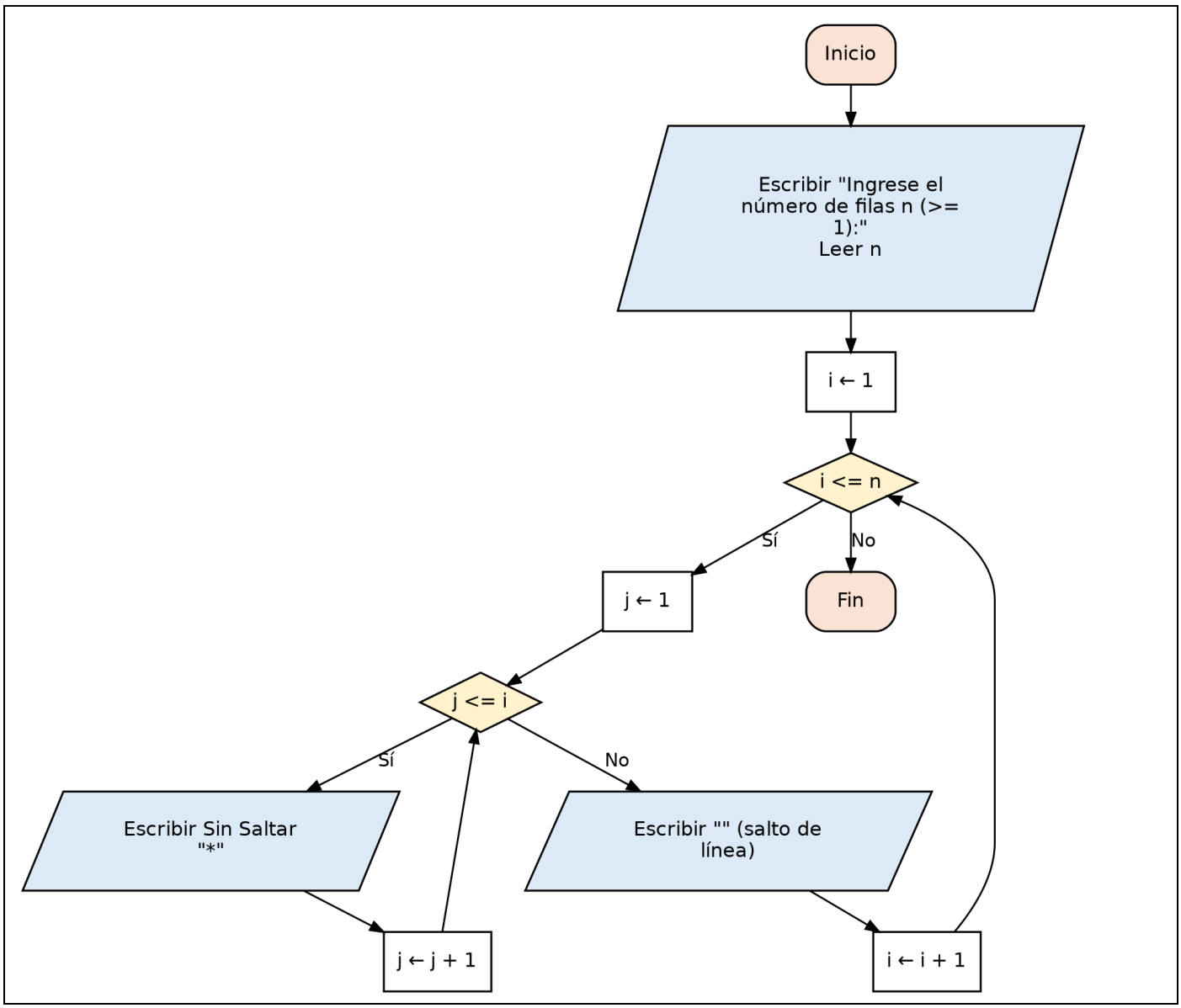
Seudocódigo

```
Proceso TrianguloAsteriscos
   Definir n, i, j Como Entero
   Escribir "Ingrese el número de filas n (>= 1):"
   Leer n
   Para i <- 1 Hasta n Hacer
      Para j <- 1 Hasta i Hacer
         Escribir Sin Saltar "*"
      FinPara
      Escribir ""
   FinPara
FinProces
```

Prueba de escritorio

i (fila)	j (columna)	Acción realizada	Estado de pantalla
1	1	Imprime *	*
2	1, 2	Imprime ** y salto de línea	*\n**
3	1, 2, 3	Imprime *** y salto de línea	*\n**\n***

Diagrama de flujo



Código fuente

```
#include <stdio.h>

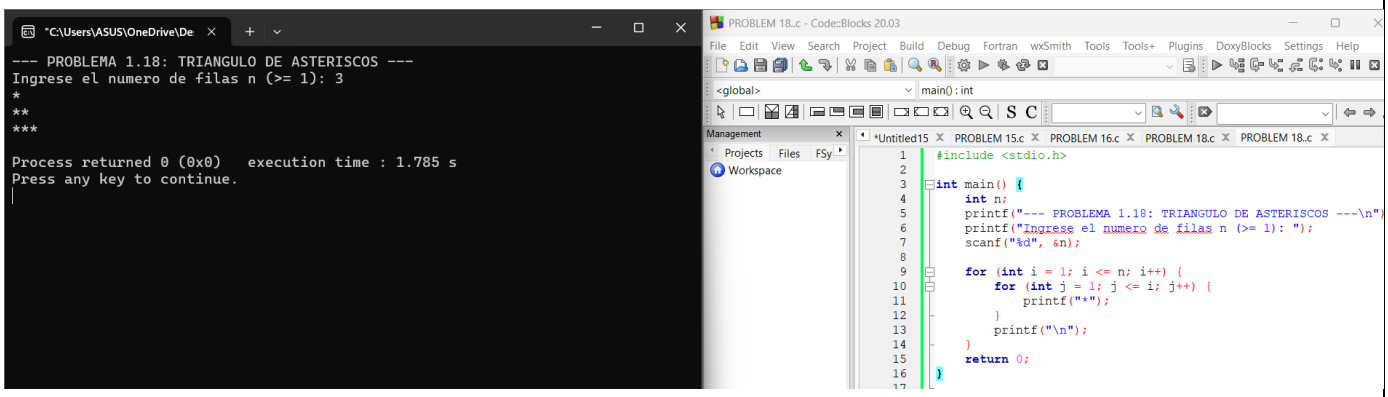
int main()
{
    int n, i, j;

    printf("Ingrese el numero de filas n (>= 1): ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++)
        {
            printf("*");
        }
        printf("\n");
```

```
    }
    return 0;
}
```

Pantallazo de Code::Blocks



19-Segundo mayor de una secuencia

Desarrolla un programa que lea números enteros positivos hasta que se introduzca un 0, y muestre el mayor y el segundo mayor de los valores ingresados. Si hay menos de dos valores, debe informarlo.

Ejemplo: 7, 3, 9, 5, 0 → mayor = 9, segundo mayor = 7.

Pista: Mantén dos variables, mayor y segundo, y actualízalas con cuidado cuando llegue un valor nuevo.

Para pensar: ¿Qué debería pasar si el valor mayor se repite (por ejemplo, 9, 9, 5)? Decide, justifica y prueba tu decisión.

Análisis del problema

- **Entrada:** Una secuencia de números enteros positivos que finaliza al introducir un 0.
- **Proceso:** Leer los números en un bucle hasta recibir un 0, actualizando y gestionando dinámicamente las variables que almacenan el valor mayor y el segundo mayor. Validar que existan al menos dos números válidos.
- **Salida:** El valor mayor y el segundo mayor de la secuencia, o un mensaje de advertencia si hay menos de dos datos.
- **Restricciones:** El número 0 marca el fin de la entrada y no se procesa como dato válido. Se requieren al menos dos valores distintos de cero.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	valor	Ingresado por teclado	Entero
Variable auxiliar	contador_elementos	0	Entero
Dato de salida	mayor	Evaluado	Entero
Dato de salida	segundo_mayor	Evaluado	Entero

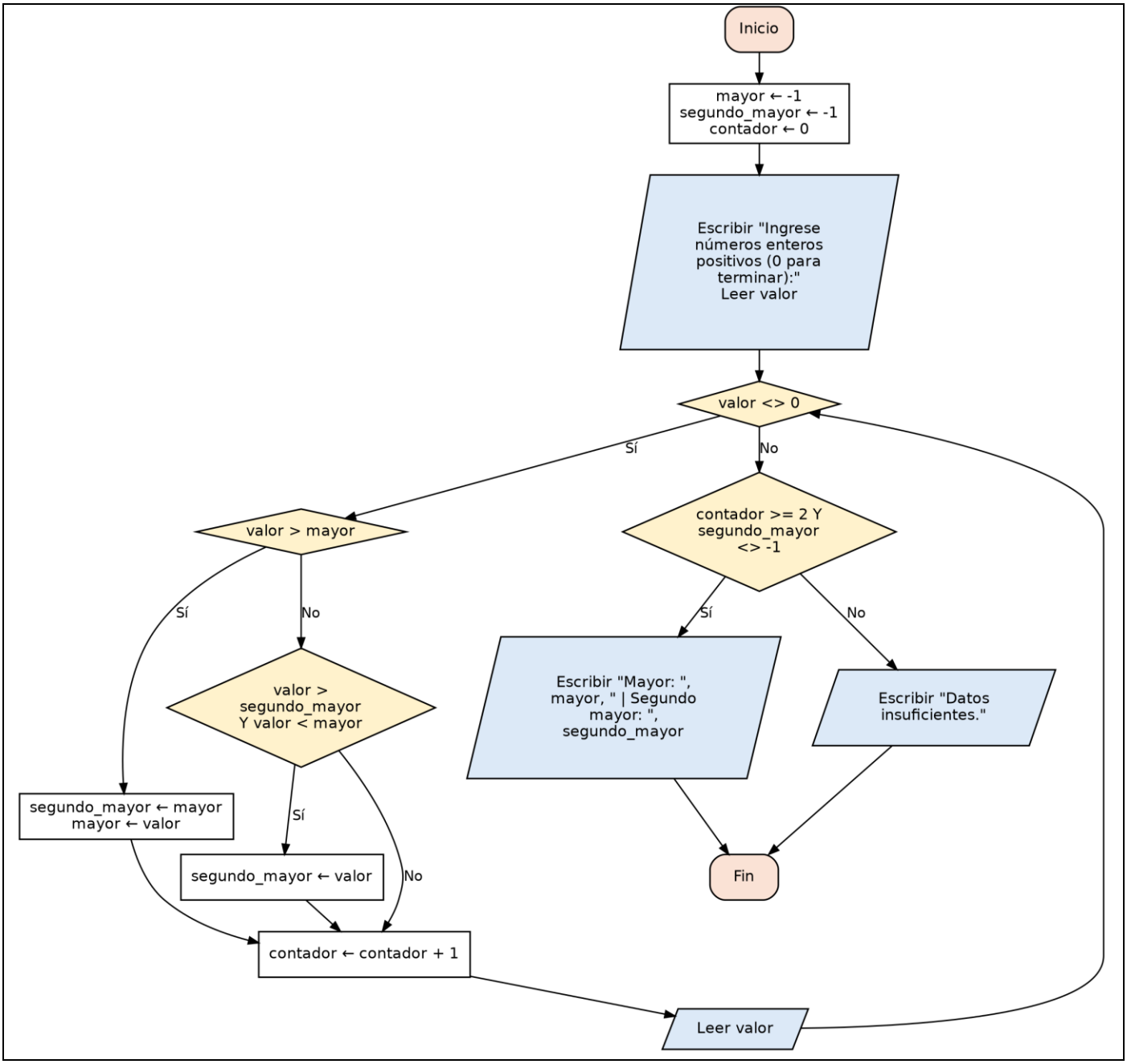
Seudocódigo

```
Proceso SegundoMayorSecuencia
  Definir valor, mayor, segundo_mayor, contador Como Entero
  mayor <- -1
  segundo_mayor <- -1
  contador <- 0
  Escribir "Ingrese números enteros positivos (0 para terminar):"
  Leer valor
  Mientras valor <> 0 Hacer
    Si valor > mayor Entonces
      segundo_mayor <- mayor
      mayor <- valor
    Sino
      Si valor > segundo_mayor Y valor < mayor Entonces
        segundo_mayor <- valor
      FinSi
    FinSi
    contador <- contador + 1
    Leer valor
  FinMientras
  Si contador >= 2 Y segundo_mayor <> -1 Entonces
    Escribir "El número mayor es: ", mayor
    Escribir "El segundo mayor es: ", segundo_mayor
  Sino
    Escribir "No se ingresaron suficientes valores válidos (se necesitan al menos dos distintos)."
  FinSi
FinProceso
```

Prueba de escritorio

valor	mayor (antes)	segundo (antes)	mayor (después)	segundo (después)
7	-1	-1	7	-1
3	7	-1	7	3
9	7	3	9	7
5	9	7	9	7
0 (Fin)	9	7	9	7 (Mayor: 9, Segundo: 7)

Diagrama de flujo



Código fuente

```
#include <stdio.h>

int main()
{
    int valor, mayor, segundo_mayor, contador;

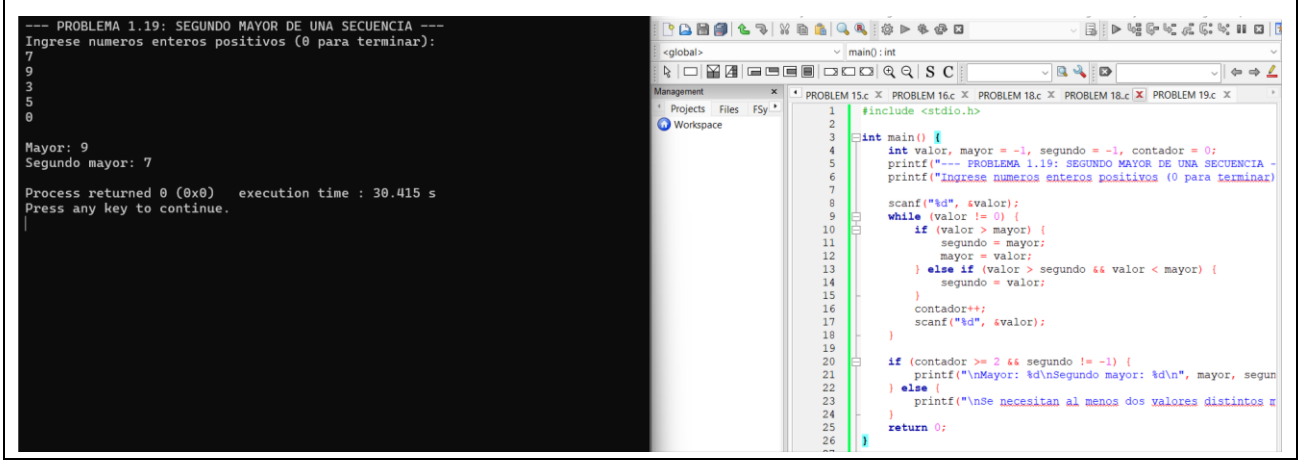
    mayor = -1;
    segundo_mayor = -1;
    contador = 0;

    printf("Ingrese numeros enteros positivos (0 para terminar):\n");
    scanf("%d", &valor);

    while (valor != 0)
    {
        if (valor > mayor)
        {
            segundo_mayor = mayor;
            mayor = valor;
        }
        else if (valor > segundo_mayor && valor < mayor)
        {
            segundo_mayor = valor;
        }
        contador = contador + 1;
        scanf("%d", &valor);
    }

    if (contador >= 2 && segundo_mayor != -1)
    {
        printf("Mayor: %d | Segundo mayor: %d\n", mayor, segundo_mayor);
    }
    else
    {
        printf("Datos insuficientes.\n");
    }
    return 0;
}
```

Pantallazo de Code::Blocks



20- Adivina el número

Desarrolla un programa que «piense» un número entre 1 y 100 (puede ser fijo para la práctica) y pida al usuario adivinarlo. Después de cada intento debe indicar «más alto», «más bajo» o «¡acertaste!», y al final mostrar cuántos intentos se usaron.

Pista: Usa un ciclo que se repita hasta que el intento sea igual al número secreto, y un contador de intentos.

Para pensar: Si jugaras con la mejor estrategia posible (dividir siempre el rango por la mitad), ¿cuál es el máximo de intentos que necesitarías para el rango 1–100? Justifica tu respuesta.

Análisis del problema

- **Entrada:** Intentos sucesivos (números enteros dentro de un rango de 1 a 100) ingresados por el usuario.
- **Proceso:** Comparar el intento del usuario con un número secreto predefinido, indicando pistas textuales ("más alto", "más bajo" o "¡acertaste!") e incrementando un contador de intentos en cada paso hasta adivinar.
- **Salida:** Mensajes de retroalimentación por cada intento, felicitación al acertar y el número total de intentos realizados.
- **Restricciones:** Los números ingresados deben mantenerse dentro del rango establecido (1 a 100).

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	intento	Ingresado por teclado	Entero
Variable auxiliar	numero_secreto	Fijo	Entero
Variable auxiliar	intentos	0	Entero
Dato de salida	mensaje_pista	Impreso en pantalla	Cadena de texto
Dato de salida	total_intentos	Valor final de intentos	Entero

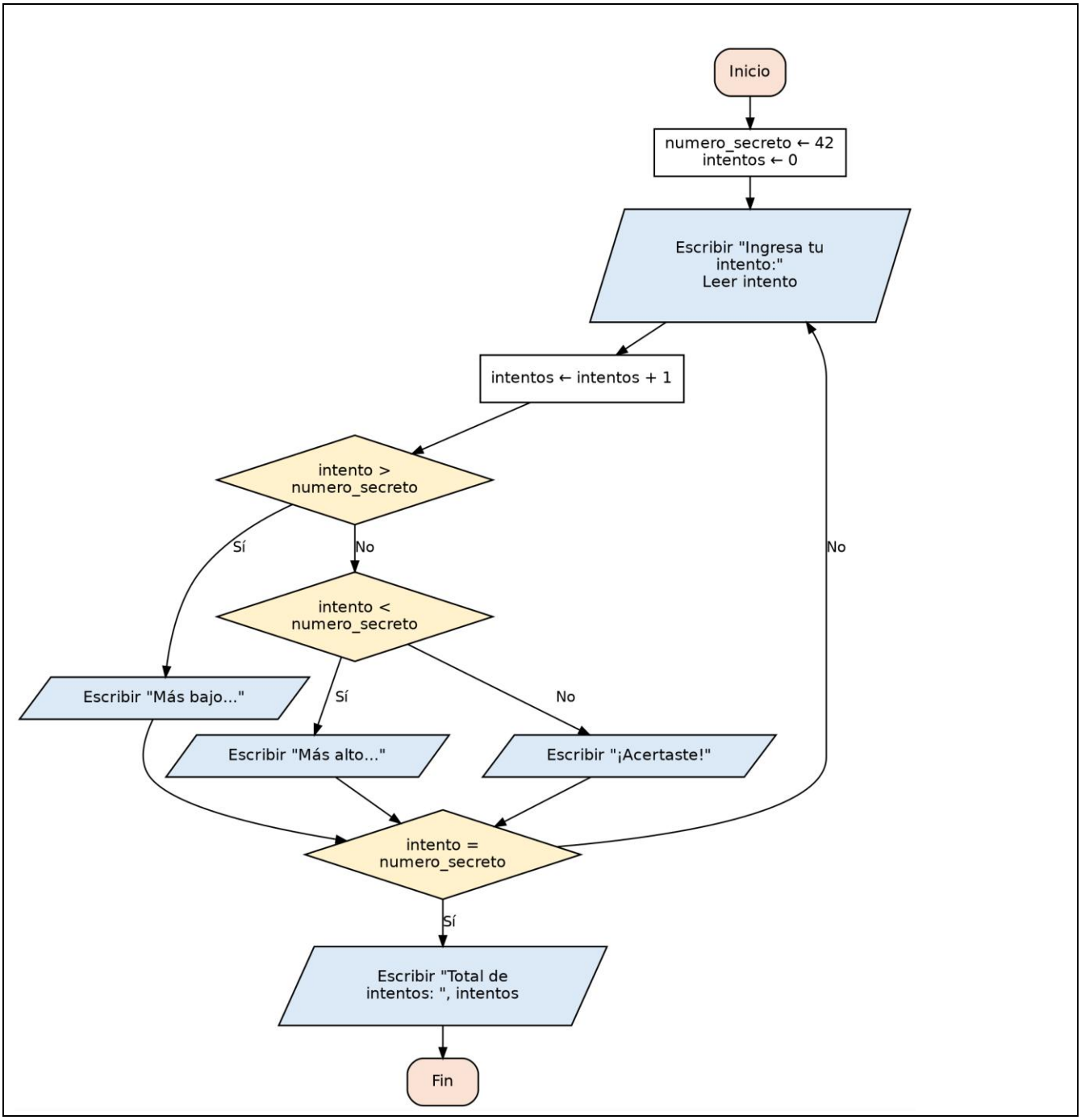
Seudocódigo

```
Proceso AdivinaElNumero
  Definir numero_secreto, intento, intentos Como Entero
  numero_secreto <- 42  // Número fijo para la práctica
  intentos <- 0
  Escribir "¡Adivina el número secreto entre 1 y 100!"
  Repetir
    Escribir "Ingresa tu intento:"
    Leer intento
    intentos <- intentos + 1
    Si intento > numero_secreto Entonces
      Escribir "Más bajo..."
    Sino
      Si intento < numero_secreto Entonces
        Escribir "Más alto..."
      Sino
        Escribir "¡Acertaste!"
      FinSi
    FinSi
  Hasta Que intento = numero_secreto
  Escribir "Utilizaste un total de ", intentos, " intentos."
FinProceso
```

Prueba de escritorio

Intento	Comparación con secreto (42)	Pista producida	Contador intentos
50	50 > 42	"Más bajo..."	1
30	30 < 42	"Más alto..."	2
42	42 == 42	"¡Acertaste!"	3 (Total intentos: 3)

Diagrama de flujo



Código fuente

```
#include <stdio.h>

int main()
{
    int numero_secreto, intento, intentos;

    numero_secreto = 42;
    intentos = 0;

    do
    {
        printf("Ingresa tu intento: ");
        scanf("%d", &intento);
        intentos = intentos + 1;

        if (intento > numero_secreto)
        {
            printf("Mas bajo...\n");
        }
        else if (intento < numero_secreto)
        {
            printf("Mas alto...\n");
        }
        else
        {
            printf("Acertaste!\n");
        }
    } while (intento != numero_secreto);

    printf("Total de intentos: %d\n", intentos);
    return 0;
}
```

Pantallazo de Code::Blocks

